VIT-AP UNIVERSITY

Introduction to Programming (Slot: F2+TF2)

C Mini Project

LuckyPaws: Animal Rescue, Pet Care and Pet Blood Donor System

Submitted by : GOLTHI SAI LAKSHMI DHEERAJA

Register No. : 25BEV7035

Submitted to : Dr A. Manimuthu

1. Introduction

Stray and injured animals often do not get help in time because people do not know where to report them. Pet owners also forget vaccination dates, and when a dog or cat needs blood there is no easy way to find a donor. LuckyPaws is a menu driven C program that keeps all of this in one place.

2. Features

- Report a stray animal, an injured animal or a missing pet, and mark an animal as rescued.
- View reports (all, open, by type or by city).
- My Pet: pet details, health records and reminders for the next 30 days.
- Pet blood donors: register donors and find compatible donors. Dogs and cats must be 1 to 8 years old, dogs at least 25 kg, cats at least 4.5 kg, and 90 days must pass between two donations.
- Food and care guide, and Ask LuckyPaws for common questions.
- All data is stored in the file luckypaws.dat and a summary can be written to luckypaws_report.txt.

3. Concepts used

- Operators, if-else, switch, loops, printf and scanf
- Arrays, 2-D arrays, strings (character arrays), functions, call by value and call by reference, recursion
- Pointers, dynamic memory allocation (malloc, calloc, realloc, free), pointer to function
- Structures, array of structures, nested structure, union, file handling (fopen, fread, fwrite, fseek, fprintf)

4. How to run

gcc luckypaws.c -o luckypaws
./luckypaws

5. Source Code

```
/*
 * LuckyPaws - Animal Rescue, Pet Care and Pet Blood Donor System
 * Name   : GOLTHI SAI LAKSHMI DHEERAJA
 * Reg No : 25BEV7035
 * Course : Introduction to Programming (F2+TF2)
 *
 * Blood donor rules used: dogs 1-8 years and 25 kg or more, cats 1-8
 * years and 4.5 kg or more, 90 days gap between two donations.
 */

#include <stdio.h>
#include <stdlib.h>

#define DATA_FILE     "luckypaws.dat"
#define REPORT_FILE   "luckypaws_report.txt"
#define MAX_RECORDS   10      /* health records kept per pet */
#define DONATION_GAP  90      /* days between two blood donations */

/* report types and status */
#define STRAY    1
#define INJURED  2
#define MISSING  3
#define OPEN     0
#define RESCUED  1
#define FOUND    2

typedef struct {
    int d, m, y;              /* y == 0 means "none" */
} Date;

/* details that depend on the report type - only one is used at a time */
typedef union {
    int  count;               /* stray   : how many animals */
    char severity[10];        /* injured : Urgent / Serious / Minor */
    char petName[30];         /* missing : the pet's name */
} ReportExtra;

typedef struct {
    int  id;
    int  type;                /* STRAY / INJURED / MISSING */
    char species[8];          /* Dog / Cat / Other */
    char area[30];
    char city[30];
    char landmark[40];
    Date date;
    char details[80];
    ReportExtra extra;
    char reporter[30];
    char phone[11];
    int  status;              /* OPEN / RESCUED / FOUND */
    char rescuer[40];
    char outcome[30];
} Report;

typedef struct {
    Date date;
    char kind[14];            /* Vaccination / Deworming / Check-up / Treatment */
    char detail[40];
    Date next;                /* next due date, y == 0 if none */
} HealthRecord;

typedef struct {
    int  id;
    char name[20];
    char species[8];
    char breed[24];
    char sex;                 /* M / F */
    Date dob;
    float weight;             /* kg */
    char colour[20];
    char microchip[16];       /* 15 digits, "" if none */
    int  recordCount;
    HealthRecord records[MAX_RECORDS];   /* array of structures inside a structure */
```

```
} Pet;

typedef struct {
    int  id;
    char name[20];
    char species[8];           /* Dog / Cat */
    int  bloodType;            /* index into DOG_TYPES or CAT_TYPES */
    Date dob;
    float weight;
    Date lastDonation;
    char owner[30];
    char phone[11];
    char city[30];
} Donor;

typedef struct {
    const char *stage;
    const char *meals;
    const char *food;
    const char *care;
} CareGuide;

typedef struct {
    const char *keys[7];       /* NULL-terminated list of keywords */
    const char *answer;
} Answer;

const char *TYPE_NAMES[4]   = {"", "Stray", "Injured", "Missing"};
const char *STATUS_NAMES[3] = {"Open", "Rescued", "Found"};
const char *DOG_TYPES[2]    = {"DEA 1 negative", "DEA 1 positive"};
const char *CAT_TYPES[3]    = {"A", "B", "AB"};

/* [donor type][patient type]: 1 = donor blood can be given to the patient */
const int DOG_COMPAT[2][2] = {
    {1, 1},                    /* DEA 1 negative: universal donor */
    {0, 1}                     /* DEA 1 positive: only to DEA 1 positive */
};
const int CAT_COMPAT[3][3] = {
    {1, 0, 0},                 /* cats: same type only - no universal donor */
    {0, 1, 0},
    {0, 0, 1}
};

/* [species: 0 dog, 1 cat][life stage] */
const CareGuide CARE[2][3] = {
    {
        {"Puppy (under 1 year)", "3-4 small meals a day (3 meals after 6 months)",
         "Good-quality puppy food; soaked kibble for very young puppies; fresh water always",
         "Vaccination course from 6-8 weeks as the vet advises; deworm every 2-3 weeks until 3
months"},
        {"Adult (1-7 years)", "2 meals a day",
         "Balanced adult dog food; rice with boiled chicken or egg as an occasional home meal",
         "Yearly vaccination boosters; deworm every 3 months; daily walks; tick and flea
prevention"},
        {"Senior (8+ years)", "2 smaller meals a day",
         "Senior dog food, lower in calories; soft food if teeth are weak",
         "Vet check-up every 6 months; gentle, shorter walks; watch joints and weight"}
    },
    {
        {"Kitten (under 1 year)", "3-4 small meals a day",
         "Kitten food (wet and dry); fresh water always",
         "Vaccination course from 8 weeks as the vet advises; deworm as advised; litter training"},
        {"Adult (1-10 years)", "2 meals a day",
         "Balanced adult cat food - cats need meat-based protein; some wet food for water",
         "Yearly vaccination boosters; regular grooming; clean the litter box daily"},
        {"Senior (11+ years)", "2-3 smaller meals a day",
         "Senior cat food; plenty of water - older cats dehydrate easily",
         "Vet check-up every 6 months; watch for weight loss; soft, warm resting places"}
    }
};
const char *UNSAFE[2] = {
    "chocolate, grapes and raisins, onion and garlic, xylitol (sugar-free sweets), cooked bones,
alcohol",
    "lilies (all parts), onion and garlic, chocolate, grapes and raisins, cow's milk, dog food as a
diet"
};
```

```
#define NUM_ANSWERS 9
const Answer ANSWERS[NUM_ANSWERS] = {
    {{"emergency", "bleeding", "accident", "poison", "seizure", "hit by", NULL},
     "This sounds like an emergency. Take the animal to the nearest vet right away and keep\n  it
calm and warm. You can also file a report with menu option 2."},
    {{"vaccin", "rabies", "booster", "injection", NULL},
     "Puppies and kittens start vaccinations at about 6-8 weeks, then boosters as the vet\n
advises. Adults usually need yearly boosters, including anti-rabies. Save them in My Pet."},
    {{"fever", "vomit", "diarrh", "not eating", "sick", NULL},
     "A pet that stops eating, keeps vomiting or is very weak should see a vet soon.\n  Never give
human medicines - many are poisonous to pets."},
    {{"chocolate", "grape", "onion", "garlic", "toxic", "lily", NULL},
     "Never give chocolate, grapes, onion, garlic or xylitol. Lilies are very dangerous\n  for cats.
If eaten, contact a vet immediately. See menu option 8."},
    {{"food", "feed", "diet", "meal", "eat", NULL},
     "Puppies and kittens need 3-4 small meals a day; adults usually 2. Always keep fresh\n  water
out. Menu option 8 has guides by age."},
    {{"blood", "donor", "donate", "transfusion", NULL},
     "Dogs 1-8 years and 25 kg+, and cats 1-8 years and 4.5 kg+, can usually donate if\n  healthy.
DEA 1-negative dogs are universal donors; cats need the same blood type."},
    {{"stray", "street", "adopt", NULL},
     "Report a stray with menu option 1, giving the area and a landmark. To take care of\n  one
yourself, use option 4 and record the outcome."},
    {{"lost", "missing", NULL},
     "Report your pet with menu option 3 - a clear description and where it was last seen.\n  A
microchip makes it much easier to return a lost pet."},
    {{"worm", "tick", "flea", NULL},
     "Adult pets are usually dewormed every 3 months. Check for ticks after walks and use\n  a vet-
approved tick and flea treatment."}
};

/* dynamic arrays (file scope: shared by all functions) */
Report *reports;  int reportCount, reportCap, nextReportId = 1;
Pet    *pets;     int petCount,    petCap,    nextPetId    = 1;
Donor  *donors;   int donorCount,  donorCap,  nextDonorId  = 1;
Date today;

/* file layout: version | 6 ints | reports | pets | donors */
#define FILE_VERSION 2
#define HEADER_SIZE ((long)(7 * sizeof(int)))

/* ---------------------------------------------------------------------
 *  Own character and string functions (character arrays + pointers)
 * --------------------------------------------------------------------- */

int isDigitChar(char c)
{
    return c >= '0' && c <= '9';
}

int isLetter(char c)
{
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
}

int isSpaceChar(char c)
{
    return c == ' ' || c == '\t' || c == '\n' || c == '\r';
}

char lowerCase(char c)
{
    return (c >= 'A' && c <= 'Z') ? (char)(c - 'A' + 'a') : c;
}

char upperCase(char c)
{
    return (c >= 'a' && c <= 'z') ? (char)(c - 'a' + 'A') : c;
}

/* length of a string: end pointer minus start pointer */
int strLength(const char *s)
{
    const char *p = s;
```

```
    while (*p != '\0')
        p++;
    return (int)(p - s);
}

void strCopy(char *dest, const char *src)
{
    while ((*dest++ = *src++) != '\0')
        ;
}

/* 0 when equal, otherwise the difference of the first characters that differ */
int strCompare(const char *a, const char *b)
{
    while (*a != '\0' && *a == *b) {
        a++;
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}

/* 1 when the character c appears in s */
int hasChar(const char *s, char c)
{
    for (; *s != '\0'; s++)
        if (*s == c)
            return 1;
    return 0;
}

/* 1 when word appears anywhere inside text */
int containsWord(const char *text, const char *word)
{
    int i, j;

    for (i = 0; text[i] != '\0'; i++) {
        for (j = 0; word[j] != '\0' && text[i + j] == word[j]; j++)
            ;
        if (word[j] == '\0')
            return 1;
    }
    return 0;
}

/* removes spaces at both ends */
void trim(char *s)
{
    int start = 0, i, n;

    while (s[start] != '\0' && isSpaceChar(s[start]))
        start++;
    for (i = 0; s[start + i] != '\0'; i++)      /* shift the text left */
        s[i] = s[start + i];
    s[i] = '\0';
    n = strLength(s);
    while (n > 0 && isSpaceChar(s[n - 1]))
        s[--n] = '\0';
}

int sameText(const char *a, const char *b)
{
    while (*a && *b) {
        if (lowerCase(*a) != lowerCase(*b))
            return 0;
        a++;
        b++;
    }
    return *a == *b;
}

void toLower(const char *in, char *out)
{
    while (*in)
        *out++ = lowerCase(*in++);
    *out = '\0';
}
```

```
/* ---------------------------------------------------------------
 *  Input helpers (fgets based)
 * --------------------------------------------------------------- */

void readLine(const char *prompt, char *buf, int size)
{
    printf("%s", prompt);
    fflush(stdout);
    if (fgets(buf, size, stdin) == NULL) {
        printf("\nInput ended. Exiting.\n");
        exit(0);
    }
    if (!hasChar(buf, '\n')) {
        int c;
        while ((c = getchar()) != '\n' && c != EOF)
            ;
    }
    trim(buf);
}

int readInt(const char *prompt, int min, int max)
{
    char buf[32], extra;
    int v;

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        /* exactly one number and nothing after it */
        if (sscanf(buf, "%d%c", &v, &extra) == 1 && v >= min && v <= max)
            return v;
        printf("  ! Please enter a number between %d and %d.\n", min, max);
    }
}

float readFloat(const char *prompt, float min, float max)
{
    char buf[32], extra;
    float v;

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        if (sscanf(buf, "%f%c", &v, &extra) == 1 && v >= min && v <= max)
            return v;
        printf("  ! Please enter a value between %.1f and %.1f.\n", min, max);
    }
}

int readYesNo(const char *prompt)
{
    char buf[8];

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        if (buf[0] == 'y' || buf[0] == 'Y')
            return 1;
        if (buf[0] == 'n' || buf[0] == 'N')
            return 0;
        printf("  ! Please answer y or n.\n");
    }
}

/* any text of at least minLen characters (an empty answer is allowed when minLen is 0) */
void readText(const char *prompt, char *buf, int size, int minLen)
{
    for (;;) {
        readLine(prompt, buf, size);
        if ((int)strLength(buf) >= minLen)
            return;
        printf("  ! Please enter at least %d characters.\n", minLen);
    }
}

/* letters, spaces and dots only */
void readWords(const char *prompt, char *buf, int size)
{
```

```
    for (;;) {
        int ok, i;

        readLine(prompt, buf, size);
        ok = buf[0] != '\0';
        for (i = 0; buf[i] && ok; i++)
            if (!isLetter(buf[i]) && buf[i] != ' ' && buf[i] != '.')
                ok = 0;
        if (ok)
            return;
        printf("  ! Required. Use letters and spaces only.\n");
    }
}

void readPhone(const char *prompt, char *buf)
{
    char tmp[32];

    for (;;) {
        int ok, i;

        readLine(prompt, tmp, sizeof tmp);
        ok = strLength(tmp) == 10 && tmp[0] >= '6' && tmp[0] <= '9';
        for (i = 0; tmp[i] && ok; i++)
            if (!isDigitChar(tmp[i]))
                ok = 0;
        if (ok) {
            strCopy(buf, tmp);
            return;
        }
        printf("  ! Enter a valid 10-digit mobile number (starts with 6-9).\n");
    }
}

/* Dog / Cat, plus Other when allowOther is set (any letter case) */
void readSpecies(const char *prompt, char *out, int allowOther)
{
    char buf[16];

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        if (sameText(buf, "dog"))                { strCopy(out, "Dog");   return; }
        if (sameText(buf, "cat"))                { strCopy(out, "Cat");   return; }
        if (allowOther && sameText(buf, "other")) { strCopy(out, "Other"); return; }
        printf("  ! Type %s.\n", allowOther ? "Dog, Cat or Other" : "Dog or Cat");
    }
}

/* ---------------------------------------------------------------------
 *  Date logic
 * --------------------------------------------------------------------- */

int isLeap(int y)
{
    return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
}

int isValidDate(Date dt)
{
    const int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    if (dt.y < 1990 || dt.y > 2100 || dt.m < 1 || dt.m > 12 || dt.d < 1)
        return 0;
    return dt.d <= ((dt.m == 2 && isLeap(dt.y)) ? 29 : days[dt.m - 1]);
}

long dateToDays(Date dt)
{
    int y = dt.y, m = dt.m;

    if (m < 3) {
        y--;
        m += 12;
    }
    return 365L * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + dt.d;
}
```

```
int ageOn(Date dob, Date on)
{
    int age = on.y - dob.y;

    if (on.m < dob.m || (on.m == dob.m && on.d < dob.d))
        age--;
    return age;
}

int compareDates(Date a, Date b)
{
    long diff = dateToDays(a) - dateToDays(b);
    return diff < 0 ? -1 : diff > 0 ? 1 : 0;
}

void formatDate(Date dt, char *out)
{
    if (dt.y == 0)
        strCopy(out, "-");
    else
        sprintf(out, "%02d/%02d/%04d", dt.d, dt.m, dt.y);
}

Date makeDate(int d, int m, int y)
{
    Date dt;

    dt.d = d;
    dt.m = m;
    dt.y = y;
    return dt;
}

/*
 * dd/mm/yyyy. "0" means none when allowNone is set.
 * future = 1 allows future dates only (due dates), 0 allows past dates only.
 */
Date readDate(const char *prompt, int allowNone, int future)
{
    char buf[32];
    Date dt;

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        if (allowNone && strCompare(buf, "0") == 0)
            return makeDate(0, 0, 0);
        if (sscanf(buf, "%d/%d/%d", &dt.d, &dt.m, &dt.y) == 3 && isValidDate(dt)) {
            if (!future && compareDates(dt, today) <= 0)
                return dt;
            if (future && compareDates(dt, today) > 0)
                return dt;
            printf("  ! The date must be %s.\n", future ? "after today" : "today or earlier");
        } else {
            printf("  ! Invalid date. Use dd/mm/yyyy%s.\n", allowNone ? ", or 0 for none" : "");
        }
    }
}

/* today's date is entered once when the program starts */
Date readToday(void)
{
    char buf[32];
    Date dt;

    for (;;) {
        readLine("Enter today's date (dd/mm/yyyy): ", buf, sizeof buf);
        if (sscanf(buf, "%d/%d/%d", &dt.d, &dt.m, &dt.y) == 3 && isValidDate(dt) && dt.y >= 2024)
            return dt;
        printf("  ! Invalid date. Use dd/mm/yyyy, e.g. 25/10/2026.\n");
    }
}

/* ---------------------------------------------------------------
 *  Dynamic arrays and recursive search
 * --------------------------------------------------------------- */
```

```
void outOfMemory(void)
{
    printf("Out of memory!\n");
    exit(1);
}

/* each array grows by doubling: malloc the first block, realloc after that */
void growReports(void)
{
    Report *bigger;

    if (reportCount < reportCap)
        return;
    reportCap = reportCap ? reportCap * 2 : 8;
    bigger = reports == NULL ? malloc(reportCap * sizeof(Report)) : realloc(reports, reportCap *
sizeof(Report));
    if (bigger == NULL)
        outOfMemory();
    reports = bigger;
}

void growPets(void)
{
    Pet *bigger;

    if (petCount < petCap)
        return;
    petCap = petCap ? petCap * 2 : 8;
    bigger = pets == NULL ? malloc(petCap * sizeof(Pet)) : realloc(pets, petCap * sizeof(Pet));
    if (bigger == NULL)
        outOfMemory();
    pets = bigger;
}

void growDonors(void)
{
    Donor *bigger;

    if (donorCount < donorCap)
        return;
    donorCap = donorCap ? donorCap * 2 : 8;
    bigger = donors == NULL ? malloc(donorCap * sizeof(Donor)) : realloc(donors, donorCap *
sizeof(Donor));
    if (bigger == NULL)
        outOfMemory();
    donors = bigger;
}

/* recursive binary search over the ids (each list is always sorted by id) */
int findReport(int low, int high, int id)
{
    int mid;

    if (low > high)
        return -1;
    mid = low + (high - low) / 2;
    if (reports[mid].id == id)
        return mid;
    return reports[mid].id < id ? findReport(mid + 1, high, id) : findReport(low, mid - 1, id);
}

int findPet(int low, int high, int id)
{
    int mid;

    if (low > high)
        return -1;
    mid = low + (high - low) / 2;
    if (pets[mid].id == id)
        return mid;
    return pets[mid].id < id ? findPet(mid + 1, high, id) : findPet(low, mid - 1, id);
}

/* ---------------------------------------------------------------
 *  Blood donor rules
```

```
 * ---------------------------------------------------------------- */

const char *bloodTypeName(const Donor *d)
{
    return strCompare(d->species, "Dog") == 0 ? DOG_TYPES[d->bloodType] : CAT_TYPES[d->bloodType];
}

int canGive(const char *species, int donorType, int patientType)
{
    if (strCompare(species, "Dog") == 0)
        return DOG_COMPAT[donorType][patientType];
    return CAT_COMPAT[donorType][patientType];
}

/* 0 = eligible, > 0 = days to wait, -1 = not eligible (*why says why) */
int donorWait(const Donor *d, const char **why)
{
    int isDog = strCompare(d->species, "Dog") == 0;
    int age = ageOn(d->dob, today);
    long since;

    *why = "";
    if (age < 1)                              { *why = "under 1 year";        return -1; }
    if (age > 8)                              { *why = "over 8 years";        return -1; }
    if (isDog && d->weight < 25.0f)           { *why = "dog under 25 kg";     return -1; }
    if (!isDog && d->weight < 4.5f)           { *why = "cat under 4.5 kg";    return -1; }
    if (d->lastDonation.y == 0)
        return 0;
    since = dateToDays(today) - dateToDays(d->lastDonation);
    if (since >= DONATION_GAP)
        return 0;
    *why = "90-day gap";
    return (int)(DONATION_GAP - since);
}

/* asks for a blood type that suits the species */
int readBloodType(const char *species)
{
    if (strCompare(species, "Dog") == 0) {
        printf("  Blood type: 1. DEA 1 negative   2. DEA 1 positive\n");
        return readInt("  Choose (1-2): ", 1, 2) - 1;
    }
    printf("  Blood type: 1. A   2. B   3. AB\n");
    return readInt("  Choose (1-3): ", 1, 3) - 1;
}

/* ----------------------------------------------------------------
 *  File handling
 * ---------------------------------------------------------------- */

int saveAll(void)
{
    FILE *fp = fopen(DATA_FILE, "wb");
    int header[7];

    if (fp == NULL) {
        printf("  ! Could not save to %s\n", DATA_FILE);
        return 0;
    }
    header[0] = FILE_VERSION;
    header[1] = reportCount; header[2] = petCount; header[3] = donorCount;
    header[4] = nextReportId; header[5] = nextPetId; header[6] = nextDonorId;
    fwrite(header, sizeof(int), 7, fp);
    fwrite(reports, sizeof(Report), reportCount, fp);
    fwrite(pets, sizeof(Pet), petCount, fp);
    fwrite(donors, sizeof(Donor), donorCount, fp);
    fclose(fp);
    return 1;
}

/* random access: rewrites one report in place after a rescue */
void saveReport(int idx)
{
    FILE *fp = fopen(DATA_FILE, "r+b");

    if (fp == NULL) {
```

```
        saveAll();
        return;
    }
    fseek(fp, HEADER_SIZE + (long)idx * (long)sizeof(Report), SEEK_SET);
    fwrite(&reports[idx], sizeof(Report), 1, fp);
    fclose(fp);
}

int loadAll(void)
{
    FILE *fp = fopen(DATA_FILE, "rb");
    int header[7];

    if (fp == NULL)
        return 0;
    /* the file starts with a version number: a file from an older program is not used */
    if (fread(header, sizeof(int), 7, fp) != 7 || header[0] != FILE_VERSION ||
        header[1] < 0 || header[2] < 0 || header[3] < 0) {
        printf("  ! %s is from an older version or damaged - starting with sample data.\n",
DATA_FILE);
        fclose(fp);
        return 0;
    }
    reportCount = header[1]; petCount = header[2]; donorCount = header[3];
    nextReportId = header[4]; nextPetId = header[5]; nextDonorId = header[6];
    reportCap = reportCount + 8; petCap = petCount + 8; donorCap = donorCount + 8;
    reports = calloc(reportCap, sizeof(Report));
    pets    = calloc(petCap, sizeof(Pet));
    donors  = calloc(donorCap, sizeof(Donor));
    if (reports == NULL || pets == NULL || donors == NULL)
        outOfMemory();
    if ((int)fread(reports, sizeof(Report), reportCount, fp) != reportCount ||
        (int)fread(pets, sizeof(Pet), petCount, fp) != petCount ||
        (int)fread(donors, sizeof(Donor), donorCount, fp) != donorCount) {
        fclose(fp);
        return 0;
    }
    fclose(fp);
    return 1;
}

/* ---------------------------------------------------------------------
 *  Sample data for the first run
 * --------------------------------------------------------------------- */

Report *newReport(int type, const char *species, const char *area, const char *city,
                  const char *landmark, Date date, const char *details,
                  const char *reporter, const char *phone)
{
    Report *r;
    Report blankReport = {0};

    growReports();
    r = &reports[reportCount++];
    *r = blankReport;
    r->id = nextReportId++;
    r->type = type;
    strCopy(r->species, species);
    strCopy(r->area, area);
    strCopy(r->city, city);
    strCopy(r->landmark, landmark);
    r->date = date;
    strCopy(r->details, details);
    strCopy(r->reporter, reporter);
    strCopy(r->phone, phone);
    r->status = OPEN;
    return r;
}

Pet *newPet(const char *name, const char *species, const char *breed, char sex,
            Date dob, float weight, const char *colour, const char *chip)
{
    Pet *p;
    Pet blankPet = {0};

    growPets();
```

```
    p = &pets[petCount++];
    *p = blankPet;
    p->id = nextPetId++;
    strCopy(p->name, name);
    strCopy(p->species, species);
    strCopy(p->breed, breed);
    p->sex = sex;
    p->dob = dob;
    p->weight = weight;
    strCopy(p->colour, colour);
    strCopy(p->microchip, chip);
    return p;
}

void addRecord(Pet *p, Date date, const char *kind, const char *detail, Date next)
{
    HealthRecord *h = &p->records[p->recordCount++];

    h->date = date;
    strCopy(h->kind, kind);
    strCopy(h->detail, detail);
    h->next = next;
}

void newDonor(const char *name, const char *species, int type, Date dob, float weight,
                     Date last, const char *owner, const char *phone, const char *city)
{
    Donor *d;
    Donor blankDonor = {0};

    growDonors();
    d = &donors[donorCount++];
    *d = blankDonor;
    d->id = nextDonorId++;
    strCopy(d->name, name);
    strCopy(d->species, species);
    d->bloodType = type;
    d->dob = dob;
    d->weight = weight;
    d->lastDonation = last;
    strCopy(d->owner, owner);
    strCopy(d->phone, phone);
    strCopy(d->city, city);
}

void loadSampleData(void)
{
    Report *r;
    Pet *p;
    Date none = makeDate(0, 0, 0);

    r = newReport(INJURED, "Dog", "Benz Circle", "Vijayawada", "Near bus stop", makeDate(24, 10,
2026),
                  "Limping on back left leg, wound on paw", "Kavya", "9876500011");
    strCopy(r->extra.severity, "Urgent");
    r = newReport(STRAY, "Cat", "Brodipet", "Guntur", "4th line market", makeDate(22, 10, 2026),
                  "Mother cat with 3 kittens, looks hungry", "Rahul", "9876500012");
    r->extra.count = 4;
    r = newReport(MISSING, "Dog", "Labbipet", "Vijayawada", "Near park gate", makeDate(20, 10,
2026),
                  "Brown Labrador, red collar", "Sneha", "9876500013");
    strCopy(r->extra.petName, "Bruno");
    r = newReport(STRAY, "Dog", "Thullur Road", "Amaravati", "Petrol bunk", makeDate(18, 10, 2026),
                  "Friendly puppy, about 3 months old", "Arjun", "9876500014");
    r->extra.count = 1;
    r->status = RESCUED;
    strCopy(r->rescuer, "Paws Shelter Amaravati");
    strCopy(r->outcome, "Taken to shelter");
    r = newReport(INJURED, "Cat", "Kothapet", "Guntur", "Behind temple", makeDate(15, 10, 2026),
                  "Eye infection", "Divya", "9876500015");
    strCopy(r->extra.severity, "Serious");
    r->status = RESCUED;
    strCopy(r->rescuer, "Dr. Ramana (vet)");
    strCopy(r->outcome, "Treated at vet clinic");

    p = newPet("Coco", "Dog", "Indie", 'F', makeDate(12, 6, 2022), 18.0f, "Fawn", "");
```

```
    addRecord(p, makeDate(10, 8, 2026), "Vaccination", "Anti-rabies booster", makeDate(10, 8,
2027));
    addRecord(p, makeDate(15, 9, 2026), "Deworming", "Tablet given", makeDate(15, 12, 2026));
    p = newPet("Simba", "Cat", "Persian", 'M', makeDate(3, 2, 2021), 5.2f, "White",
"985112003456789");
    addRecord(p, makeDate(20, 7, 2026), "Vaccination", "FVRCP", makeDate(20, 7, 2027));
    p = newPet("Max", "Dog", "German Shepherd", 'M', makeDate(25, 11, 2020), 34.0f, "Black and tan",
"");
    addRecord(p, makeDate(1, 3, 2026), "Vaccination", "DHPPi + Lepto", makeDate(5, 11, 2026));

    newDonor("Max", "Dog", 0, makeDate(25, 11, 2020), 34.0f, makeDate(1, 6, 2026), "Vamsi",
"9876501234", "Amaravati");
    newDonor("Rocky", "Dog", 1, makeDate(10, 4, 2021), 30.0f, makeDate(20, 9, 2026), "Kiran",
"9123400001", "Vijayawada");
    newDonor("Simba", "Cat", 0, makeDate(3, 2, 2021), 5.2f, none, "Lakshmi", "9000012345",
"Guntur");
    newDonor("Bella", "Dog", 0, makeDate(15, 1, 2017), 28.0f, none, "Priya", "8877665544",
"Guntur");
    newDonor("Luna", "Cat", 1, makeDate(8, 8, 2022), 4.8f, none, "Anjali", "6301234567",
"Vijayawada");
}

/* ------------------------------------------------------------------
 *  Display helpers
 * ------------------------------------------------------------------ */

void printLine(void)
{
    printf("----------------------------------------------------------------------------------------------
------\n");
}

void extraText(const Report *r, char *out)
{
    switch (r->type) {
    case STRAY:   sprintf(out, "%d animal(s)", r->extra.count);    break;
    case INJURED: sprintf(out, "%s", r->extra.severity);           break;
    default:      sprintf(out, "Pet: %s", r->extra.petName);       break;
    }
}

void printReport(const Report *r)
{
    char date[16], extra[40];

    formatDate(r->date, date);
    extraText(r, extra);
    printf("%-3d %-8s %-6s %-14.14s %-11.11s %-10s %-14.14s %s\n",
           r->id, TYPE_NAMES[r->type], r->species, r->area, r->city, date, extra, STATUS_NAMES[r-
>status]);
    printf("    %s", r->details);
    if (r->status != OPEN)
        printf("  [%s: %s]", r->rescuer, r->outcome);
    printf("\n");
}

void printReportHeader(void)
{
    printLine();
    printf("%-3s %-8s %-6s %-14s %-11s %-10s %-14s %s\n", "ID", "Type", "Animal", "Area", "City",
"Date", "Details", "Status");
    printLine();
}

void printDonor(const Donor *d)
{
    char status[32];
    const char *why;
    int wait = donorWait(d, &why);

    if (wait == 0)
        strCopy(status, "Eligible");
    else if (wait > 0)
        sprintf(status, "Wait %d days", wait);
    else
        sprintf(status, "Not eligible (%s)", why);
```

```
    printf("%-3d %-8s %-4s %-15s %-4d %5.1f  %-11s %-16.16s %-10s %s\n",
           d->id, d->name, d->species, bloodTypeName(d), ageOn(d->dob, today), d->weight,
           d->city, d->owner, d->phone, status);
}

void printDonorHeader(void)
{
    printLine();
    printf("%-3s %-8s %-4s %-15s %-4s %5s  %-11s %-16s %-10s %s\n",
           "ID", "Pet", "Sp.", "Blood type", "Age", "Kg", "City", "Owner", "Phone", "Status");
    printLine();
}

/* ------------------------------------------------------------------
 *  Sorting (the compare rule is passed as a pointer to a function)
 * ------------------------------------------------------------------ */

int byDateNewest(const Report *x, const Report *y)
{
    int c = compareDates(y->date, x->date);
    return c ? c : y->id - x->id;
}

int byCity(const Report *x, const Report *y)
{
    int c = strCompare(x->city, y->city);
    return c ? c : compareDates(y->date, x->date);
}

/* insertion sort of an array of report pointers, using any compare function */
void sortReports(const Report *view[], int n, int (*compare)(const Report *, const Report *))
{
    int i, j;

    for (i = 1; i < n; i++) {
        const Report *key = view[i];

        for (j = i - 1; j >= 0 && compare(view[j], key) > 0; j--)
            view[j + 1] = view[j];
        view[j + 1] = key;
    }
}

/* ------------------------------------------------------------------
 *  Rescue & Stray
 * ------------------------------------------------------------------ */

void fileReport(int type)
{
    Report r = {0};          /* every member starts at zero */
    const char *detailsPrompt = type == INJURED ? "Describe the injury" :
                                type == MISSING ? "Breed, colour, collar and any marks" :
                                                  "Condition (hungry, friendly, with young...)";

    printf("\n--- Report %s ---\n", type == STRAY ? "a Stray Animal" : type == INJURED ? "an Injured
Animal" : "a Missing Pet");
    if (type == INJURED)
        printf("If the animal is bleeding heavily or cannot move, call a vet first.\n");

    readSpecies("Animal (Dog/Cat/Other)    : ", r.species, 1);
    switch (type) {
    case STRAY:
        r.extra.count = readInt("How many animals          : ", 1, 50);
        break;
    case INJURED:
        printf("How serious is it? 1. Urgent  2. Serious  3. Minor\n");
        switch (readInt("Choose (1-3)              : ", 1, 3)) {
        case 1: strCopy(r.extra.severity, "Urgent");  break;
        case 2: strCopy(r.extra.severity, "Serious"); break;
        default: strCopy(r.extra.severity, "Minor");  break;
        }
        break;
    default:
        readWords("Pet's name                : ", r.extra.petName, sizeof r.extra.petName);
        break;
    }
```

```
    r.date = readDate(type == MISSING ? "Last seen on (dd/mm/yyyy) : " : "Date seen (dd/mm/yyyy)
: ", 0, 0);
    readText("Area / locality           : ", r.area, sizeof r.area, 2);
    readWords("City                      : ", r.city, sizeof r.city);
    readText("Landmark (Enter to skip)  : ", r.landmark, sizeof r.landmark, 0);
    printf("%s\n", detailsPrompt);
    readText("  > ", r.details, sizeof r.details, 5);
    readWords("Your name                 : ", r.reporter, sizeof r.reporter);
    readPhone("Mobile number             : ", r.phone);

    growReports();
    r.id = nextReportId++;
    r.type = type;
    r.status = OPEN;
    reports[reportCount++] = r;
    saveAll();
    printf("\n  Report #%d submitted. Thank you for helping!\n", r.id);
}

void reportStray(void)   { fileReport(STRAY); }
void reportInjured(void) { fileReport(INJURED); }
void reportMissing(void) { fileReport(MISSING); }

void rescueAnimal(void)
{
    const char *OUTCOMES[5] = {"Taken to shelter", "Treated at vet clinic", "Adopted",
                               "Fed and released", "Returned to owner"};
    const Report *r;
    int open = 0, id, idx, i;

    printf("\n--- Rescue a Stray Animal / Mark a Pet Found ---\n");
    for (r = reports; r < reports + reportCount; r++) {      /* pointer arithmetic */
        if (r->status != OPEN)
            continue;
        if (open++ == 0)
            printReportHeader();
        printReport(r);
    }
    if (open == 0) {
        printf("  There are no open reports right now.\n");
        return;
    }
    printLine();
    id = readInt("Report ID you will take care of: ", 1, 999999);
    idx = findReport(0, reportCount - 1, id);
    if (idx < 0 || reports[idx].status != OPEN) {
        printf("  ! No open report with ID %d.\n", id);
        return;
    }
    readText("Rescuer / shelter name    : ", reports[idx].rescuer, sizeof reports[idx].rescuer, 3);
    printf("Outcome:\n");
    for (i = 0; i < 5; i++)
        printf("  %d. %s\n", i + 1, OUTCOMES[i]);
    strCopy(reports[idx].outcome, OUTCOMES[readInt("Choose (1-5)              : ", 1, 5) - 1]);
    if (!readYesNo("Confirm and save? (y/n)   : ")) {
        reports[idx].rescuer[0] = reports[idx].outcome[0] = '\0';
        printf("  Not saved.\n");
        return;
    }
    reports[idx].status = reports[idx].type == MISSING ? FOUND : RESCUED;
    saveReport(idx);
    printf("\n  Report #%d marked as %s. Well done!\n", id, STATUS_NAMES[reports[idx].status]);
}

void viewReports(void)
{
    const Report **view;
    char city[30] = "";
    int choice, shown = 0, i;
    int (*compare)(const Report *, const Report *) = byDateNewest;

    if (reportCount == 0) {
        printf("\n  No reports yet.\n");
        return;
    }
    printf("\nShow: 1. All  2. Open  3. Stray  4. Injured  5. Missing  6. One city  7. All, by
```

```
city\n");
    choice = readInt("Choice: ", 1, 7);
    if (choice == 6)
        readWords("City: ", city, sizeof city);
    if (choice == 7)
        compare = byCity;

    view = calloc(reportCount, sizeof *view);
    if (view == NULL)
        return;
    for (i = 0; i < reportCount; i++)
        view[i] = reports + i;
    sortReports(view, reportCount, compare);

    printf("\n");
    printReportHeader();
    for (i = 0; i < reportCount; i++) {
        const Report *r = view[i];

        if ((choice == 2 && r->status != OPEN) ||
            (choice >= 3 && choice <= 5 && r->type != choice - 2) ||
            (choice == 6 && !sameText(r->city, city)))
            continue;
        printReport(r);
        shown++;
    }
    printLine();
    printf("%d report(s) shown.\n", shown);
    free(view);
}

/* ----------------------------------------------------------------
 *  My Pet
 * ---------------------------------------------------------------- */

void listPets(void)
{
    const Pet *p;
    char dob[16];

    printLine();
    printf("%-3s %-8s %-6s %-16s %-3s %-11s %-4s %6s  %s\n", "ID", "Name", "Sp.", "Breed", "Sex",
"Born", "Age", "Kg", "Records");
    printLine();
    for (p = pets; p < pets + petCount; p++) {
        formatDate(p->dob, dob);
        printf("%-3d %-8s %-6s %-16.16s  %c  %-11s %-4d %6.1f  %d\n",
               p->id, p->name, p->species, p->breed, p->sex, dob, ageOn(p->dob, today), p->weight,
p->recordCount);
    }
    printLine();
}

void addPet(void)
{
    Pet p = {0};

    printf("\n--- Add a Pet ---\n");
    readWords("Pet's name                : ", p.name, sizeof p.name);
    readSpecies("Species (Dog/Cat/Other)   : ", p.species, 1);
    readText("Breed                     : ", p.breed, sizeof p.breed, 2);
    for (;;) {
        char buf[8];

        readLine("Sex (M/F)                 : ", buf, sizeof buf);
        p.sex = upperCase(buf[0]);
        if (buf[1] == '\0' && (p.sex == 'M' || p.sex == 'F'))
            break;
        printf("  ! Enter M or F.\n");
    }
    p.dob = readDate("Date of birth (dd/mm/yyyy): ", 0, 0);
    p.weight = readFloat("Weight (kg)               : ", 0.1f, 120.0f);
    readText("Colour / markings         : ", p.colour, sizeof p.colour, 0);
    for (;;) {
        int ok, i;
```

```
        readText("Microchip (Enter to skip) : ", p.microchip, sizeof p.microchip, 0);
        ok = p.microchip[0] == '\0' || strLength(p.microchip) == 15;
        for (i = 0; p.microchip[i] && ok; i++)
            if (!isDigitChar(p.microchip[i]))
                ok = 0;
        if (ok)
            break;
        printf("  ! A microchip number has 15 digits.\n");
    }
    growPets();
    p.id = nextPetId++;
    pets[petCount++] = p;
    saveAll();
    printf("\n  %s added to My Pet with ID %d.\n", p.name, p.id);
}

void showPet(const Pet *p)
{
    char dob[16], d1[16], d2[16];
    int i;

    formatDate(p->dob, dob);
    printLine();
    printf("  %s (ID %d) - %s, %s, %s\n", p->name, p->id, p->species, p->breed, p->sex == 'M' ?
"Male" : "Female");
    printf("  Born %s (%d years)   Weight %.1f kg   Colour %s\n", dob, ageOn(p->dob, today), p-
>weight,
           p->colour[0] ? p->colour : "-");
    printf("  Microchip: %s\n", p->microchip[0] ? p->microchip : "not chipped");
    printLine();
    if (p->recordCount == 0) {
        printf("  No health records yet.\n");
        return;
    }
    printf("  %-11s %-12s %-30s %s\n", "Date", "Type", "Details", "Next due");
    for (i = 0; i < p->recordCount; i++) {
        const HealthRecord *h = &p->records[i];

        formatDate(h->date, d1);
        formatDate(h->next, d2);
        printf("  %-11s %-12s %-30.30s %s\n", d1, h->kind, h->detail, d2);
    }
    printLine();
}

int askForPet(void)
{
    int id, idx;

    if (petCount == 0) {
        printf("  No pets yet.\n");
        return -1;
    }
    listPets();
    id = readInt("Pet ID: ", 1, 999999);
    idx = findPet(0, petCount - 1, id);
    if (idx < 0)
        printf("  ! No pet with ID %d.\n", id);
    return idx;
}

void addHealthRecord(void)
{
    const char *KINDS[4] = {"Vaccination", "Deworming", "Check-up", "Treatment"};
    HealthRecord h;
    Pet *p;
    int idx;

    printf("\n--- Add a Health Record ---\n");
    idx = askForPet();
    if (idx < 0)
        return;
    p = &pets[idx];
    if (p->recordCount == MAX_RECORDS) {
        printf("  ! %s already has %d records (the maximum).\n", p->name, MAX_RECORDS);
        return;
```

```
    }
    h.date = readDate("Date (dd/mm/yyyy)         : ", 0, 0);
    printf("Type: 1. Vaccination  2. Deworming  3. Check-up  4. Treatment\n");
    strCopy(h.kind, KINDS[readInt("Choose (1-4)              : ", 1, 4) - 1]);
    readText("Details                   : ", h.detail, sizeof h.detail, 3);
    h.next = readDate("Next due (dd/mm/yyyy, 0 if none): ", 1, 1);
    p->records[p->recordCount++] = h;
    saveAll();
    printf("\n  Record added for %s.\n", p->name);
}

void dueReminders(void)
{
    const Pet *p;
    int i, shown = 0;

    printf("\n--- Due in the next 30 days (or overdue) ---\n");
    for (p = pets; p < pets + petCount; p++) {
        for (i = 0; i < p->recordCount; i++) {
            const HealthRecord *h = &p->records[i];
            long days;
            char due[16];

            if (h->next.y == 0)
                continue;
            days = dateToDays(h->next) - dateToDays(today);
            if (days > 30)
                continue;
            formatDate(h->next, due);
            if (days < 0)
                printf("  %-8s %-24s %s  OVERDUE by %ld days\n", p->name, h->detail, due, -days);
            else
                printf("  %-8s %-24s %s  due in %ld days\n", p->name, h->detail, due, days);
            shown++;
        }
    }
    if (shown == 0)
        printf("  Nothing is due in the next 30 days.\n");
}

void myPetMenu(void)
{
    int idx;

    printf("\n--- My Pet ---\n");
    printf("1. View my pets  2. Pet profile & records  3. Add a pet  4. Add a health record  5.
Reminders\n");
    switch (readInt("Choice: ", 1, 5)) {
    case 1: printf("\n"); listPets(); break;
    case 2:
        idx = askForPet();
        if (idx >= 0)
            showPet(&pets[idx]);
        break;
    case 3: addPet(); break;
    case 4: addHealthRecord(); break;
    default: dueReminders(); break;
    }
}

/* ---------------------------------------------------------------------
 *  Pet blood donors
 * --------------------------------------------------------------------- */

void registerDonor(void)
{
    Donor d = {0};
    const char *why;
    int wait;

    printf("\n--- Register a Pet Blood Donor ---\n");
    printf("Rules: dogs 1-8 years and 25 kg+, cats 1-8 years and 4.5 kg+, 90 days between
donations.\n");
    readWords("Pet's name                : ", d.name, sizeof d.name);
    readSpecies("Species (Dog/Cat)         : ", d.species, 0);
    d.bloodType = readBloodType(d.species);
```

```
    d.dob = readDate("Date of birth (dd/mm/yyyy): ", 0, 0);
    d.weight = readFloat("Weight (kg)               : ", 0.5f, 120.0f);
    for (;;) {
        d.lastDonation = readDate("Last donation (dd/mm/yyyy, 0 if never): ", 1, 0);
        if (d.lastDonation.y == 0 || compareDates(d.lastDonation, d.dob) > 0)
            break;
        printf("  ! The last donation must be after the date of birth.\n");
    }
    readWords("Owner's name              : ", d.owner, sizeof d.owner);
    readPhone("Mobile number             : ", d.phone);
    readWords("City                      : ", d.city, sizeof d.city);

    growDonors();
    d.id = nextDonorId++;
    donors[donorCount++] = d;
    saveAll();
    wait = donorWait(&d, &why);
    printf("\n  %s registered as donor %d. ", d.name, d.id);
    if (wait == 0)
        printf("Eligible to donate now.\n");
    else if (wait > 0)
        printf("Can donate again in %d days.\n", wait);
    else
        printf("Not eligible yet (%s).\n", why);
}

void findDonors(void)
{
    char species[8], city[30];
    const Donor *d;
    const char *why;
    int patientType, found = 0;

    printf("\n--- Find a Compatible Donor ---\n");
    readSpecies("Patient species (Dog/Cat) : ", species, 0);
    patientType = readBloodType(species);
    readLine("City (Enter for all)      : ", city, sizeof city);

    printf("\nCompatible blood types for this patient: ");
    {
        int t, n = strCompare(species, "Dog") == 0 ? 2 : 3;

        for (t = 0; t < n; t++)
            if (canGive(species, t, patientType))
                printf("%s  ", strCompare(species, "Dog") == 0 ? DOG_TYPES[t] : CAT_TYPES[t]);
    }
    printf("\n\n");
    for (d = donors; d < donors + donorCount; d++) {
        if (strCompare(d->species, species) != 0 || !canGive(species, d->bloodType, patientType))
            continue;
        if (city[0] && !sameText(d->city, city))
            continue;
        if (donorWait(d, &why) != 0)
            continue;
        if (found++ == 0)
            printDonorHeader();
        printDonor(d);
    }
    if (found == 0)
        printf("  No eligible donor right now. Contact your nearest veterinary blood bank.\n");
    else {
        printLine();
        printf("%d eligible donor(s) found.\n", found);
    }
    if (strCompare(species, "Cat") == 0)
        printf("Note: cats have no universal donor - the vet must cross-match the blood.\n");
}

void donorMenu(void)
{
    const Donor *d;

    printf("\n--- Pet Blood Donors ---\n");
    printf("1. Find a compatible donor  2. Register a donor  3. View all donors\n");
    switch (readInt("Choice: ", 1, 3)) {
    case 1: findDonors(); break;
```

```
    case 2: registerDonor(); break;
    default:
        printf("\n");
        printDonorHeader();
        for (d = donors; d < donors + donorCount; d++)
            printDonor(d);
        printLine();
        break;
    }
}

/* ------------------------------------------------------------------
 *  Food & care, Ask LuckyPaws, export
 * ------------------------------------------------------------------ */

void careGuide(void)
{
    char species[8];
    int s, stage;
    const CareGuide *g;

    printf("\n--- Pet Food & Care ---\n");
    readSpecies("Species (Dog/Cat)         : ", species, 0);
    s = strCompare(species, "Dog") == 0 ? 0 : 1;
    printf("Life stage: 1. %s  2. %s  3. %s\n", CARE[s][0].stage, CARE[s][1].stage, CARE[s]
[2].stage);
    stage = readInt("Choose (1-3)              : ", 1, 3) - 1;
    g = &CARE[s][stage];

    printLine();
    printf("  %s - %s\n", species, g->stage);
    printLine();
    printf("  Meals      : %s\n", g->meals);
    printf("  Food       : %s\n", g->food);
    printf("  Care       : %s\n", g->care);
    printf("  Never feed : %s\n", UNSAFE[s]);
    printLine();
    printf("  General guidance only - follow your vet's advice.\n");
}

void askLuckyPaws(void)
{
    char question[120], lower[120];
    int i, k;

    printf("\n--- Ask LuckyPaws ---\n");
    printf("Ask about food, vaccinations, sickness, toxic foods, strays, missing pets or blood\n");
    printf("donation. Answers come from LuckyPaws' built-in guide - I am not a vet.\n");
    printf("Type 'back' to return to the menu.\n");
    for (;;) {
        readLine("\nYou: ", question, sizeof question);
        if (question[0] == '\0')
            continue;
        toLower(question, lower);
        if (strCompare(lower, "back") == 0)
            return;
        for (i = 0; i < NUM_ANSWERS; i++) {
            for (k = 0; ANSWERS[i].keys[k] != NULL; k++)
                if (containsWord(lower, ANSWERS[i].keys[k]))
                    break;
            if (ANSWERS[i].keys[k] != NULL)
                break;
        }
        if (i < NUM_ANSWERS)
            printf("LuckyPaws: %s\n", ANSWERS[i].answer);
        else
            printf("LuckyPaws: I don't know that one yet. Try food, vaccination, sick, toxic,\n
stray, missing or blood - or ask your vet.\n");
    }
}

void exportReport(void)
{
    FILE *fp = fopen(REPORT_FILE, "w");
    const Report *r;
    const Donor *d;
```

```
    char date[16], extra[40];
    int open = 0, eligible = 0;
    const char *why;

    if (fp == NULL) {
        printf("  ! Could not create %s\n", REPORT_FILE);
        return;
    }
    formatDate(today, date);
    fprintf(fp, "LuckyPaws - Summary Report (%s)\n\n", date);
    fprintf(fp, "Rescue & Stray reports\n");
    for (r = reports; r < reports + reportCount; r++) {
        formatDate(r->date, date);
        extraText(r, extra);
        fprintf(fp, "  #%-3d %-8s %-6s %-14.14s %-11.11s %s  %-14s %s\n",
                r->id, TYPE_NAMES[r->type], r->species, r->area, r->city, date, extra,
STATUS_NAMES[r->status]);
        if (r->status == OPEN)
            open++;
    }
    fprintf(fp, "\nPets: %d   Open reports: %d   Animals helped: %d\n", petCount, open, reportCount
- open);
    fprintf(fp, "\nPet blood donors\n");
    for (d = donors; d < donors + donorCount; d++) {
        int w = donorWait(d, &why);

        if (w == 0)
            eligible++;
        fprintf(fp, "  %-8s %-4s %-15s %-11s %s%s%s\n", d->name, d->species, bloodTypeName(d), d-
>city,
                w == 0 ? "Eligible" : w > 0 ? "Waiting (90-day gap)" : "Not eligible (",
                w < 0 ? why : "", w < 0 ? ")" : "");
    }
    fprintf(fp, "\nDonors: %d   Eligible now: %d\n", donorCount, eligible);
    fclose(fp);
    printf("\n  Summary written to %s\n", REPORT_FILE);
}

/* ---------------------------------------------------------------------
 *  Main menu (array of pointers to functions)
 * --------------------------------------------------------------------- */

void (*const MENU_ACTIONS[11])(void) = {
    NULL, reportStray, reportInjured, reportMissing, rescueAnimal, viewReports,
    myPetMenu, donorMenu, careGuide, askLuckyPaws, exportReport
};

void printMenu(void)
{
    char date[16];
    const Report *r;
    int open = 0;

    for (r = reports; r < reports + reportCount; r++)
        if (r->status == OPEN)
            open++;
    formatDate(today, date);
    printf("\n======================================================\n");
    printf("     LuckyPaws - Rescue . Care . Connect\n");
    printf("======================================================\n");
    printf(" Today: %s   Open reports: %d   Pets: %d   Donors: %d\n", date, open, petCount,
donorCount);
    printf("------------------------------------------------------\n");
    printf(" RESCUE & STRAY\n");
    printf("  1. Report a stray animal\n");
    printf("  2. Report an injured animal\n");
    printf("  3. Report a missing pet\n");
    printf("  4. Rescue a stray / mark a pet found\n");
    printf("  5. View reports\n");
    printf(" MY PET & MORE\n");
    printf("  6. My Pet (profiles, health records, reminders)\n");
    printf("  7. Pet blood donors\n");
    printf("  8. Pet food & care guide\n");
    printf("  9. Ask LuckyPaws\n");
    printf(" 10. Export summary report\n");
    printf("  0. Save and exit\n");
```

```
    printf("-----------------------------------------------------\n");
}

int main(void)
{
    int choice;

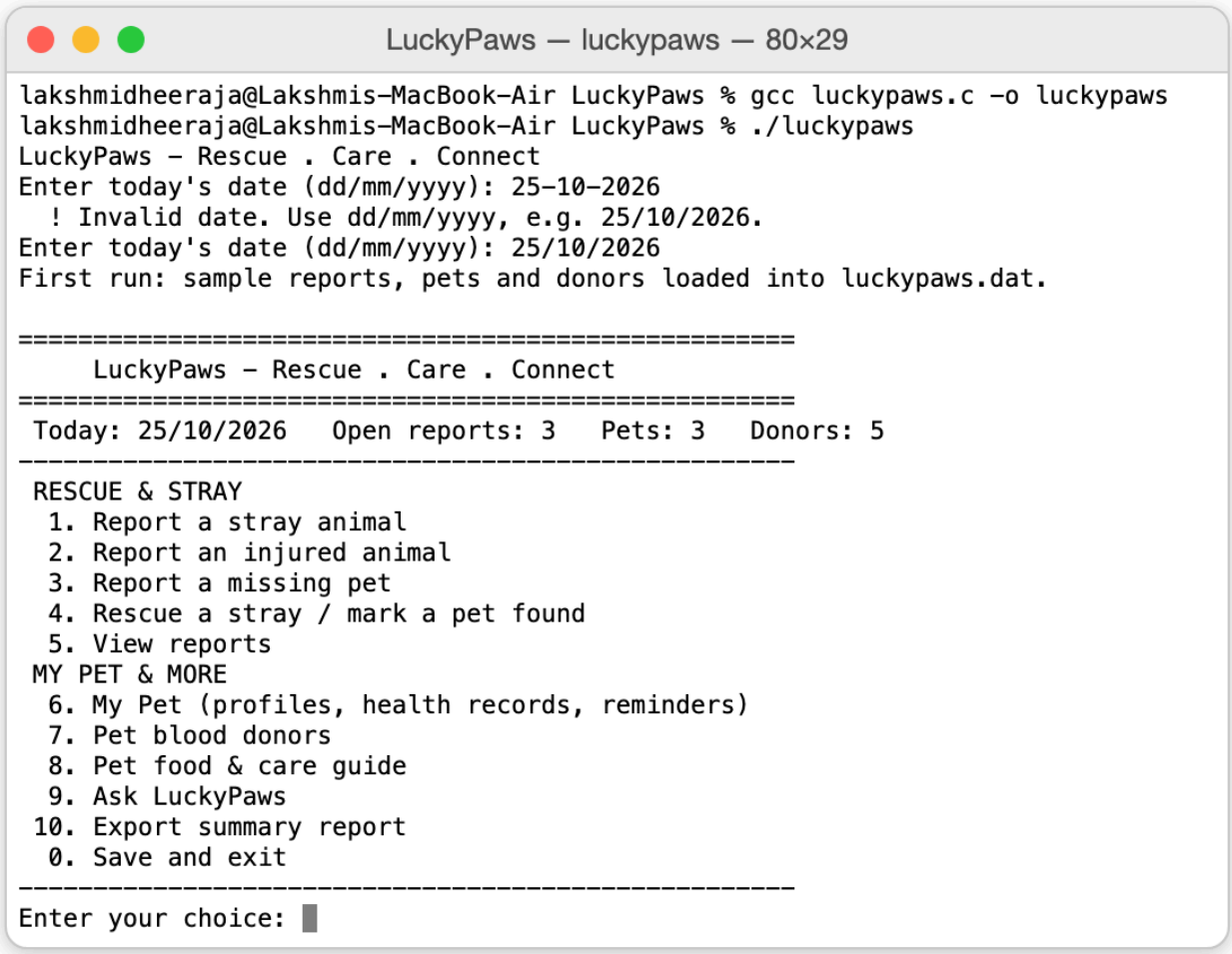
    printf("LuckyPaws - Rescue . Care . Connect\n");
    today = readToday();
    if (!loadAll()) {
        loadSampleData();
        saveAll();
        printf("First run: sample reports, pets and donors loaded into %s.\n", DATA_FILE);
    }

    do {
        printMenu();
        choice = readInt("Enter your choice: ", 0, 10);
        if (choice != 0)
            MENU_ACTIONS[choice]();          /* call through the function pointer */
    } while (choice != 0);

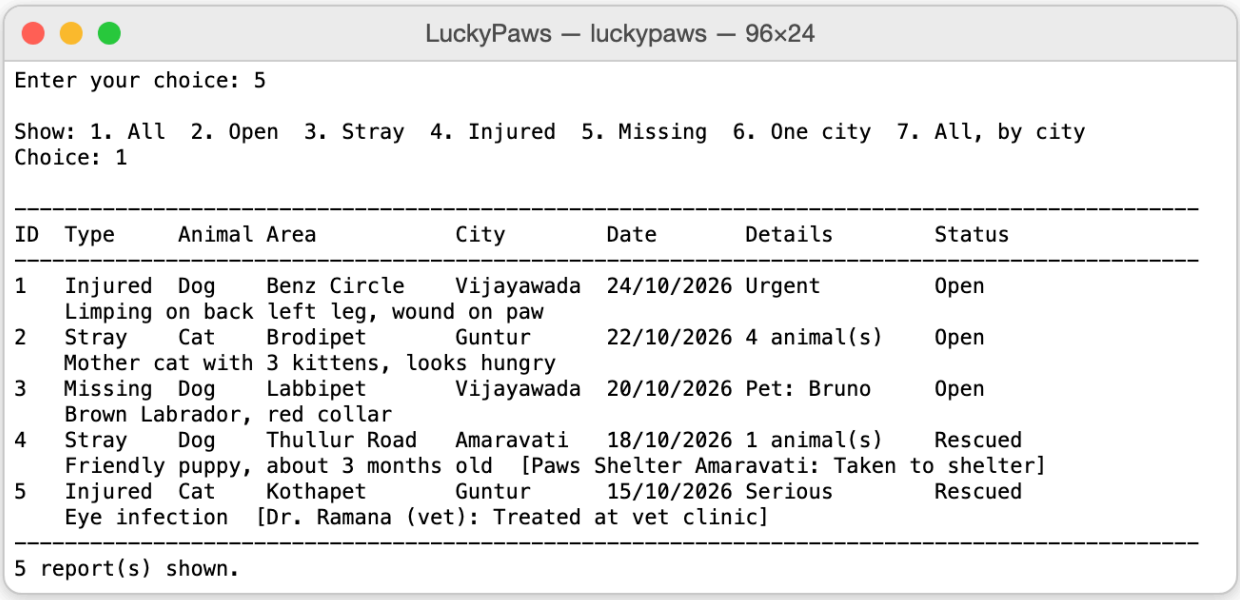
    if (saveAll())
        printf("\nData saved to %s. Thank you for caring for animals!\n", DATA_FILE);
    free(reports);
    free(pets);
    free(donors);
    return 0;
}
```

6. Output

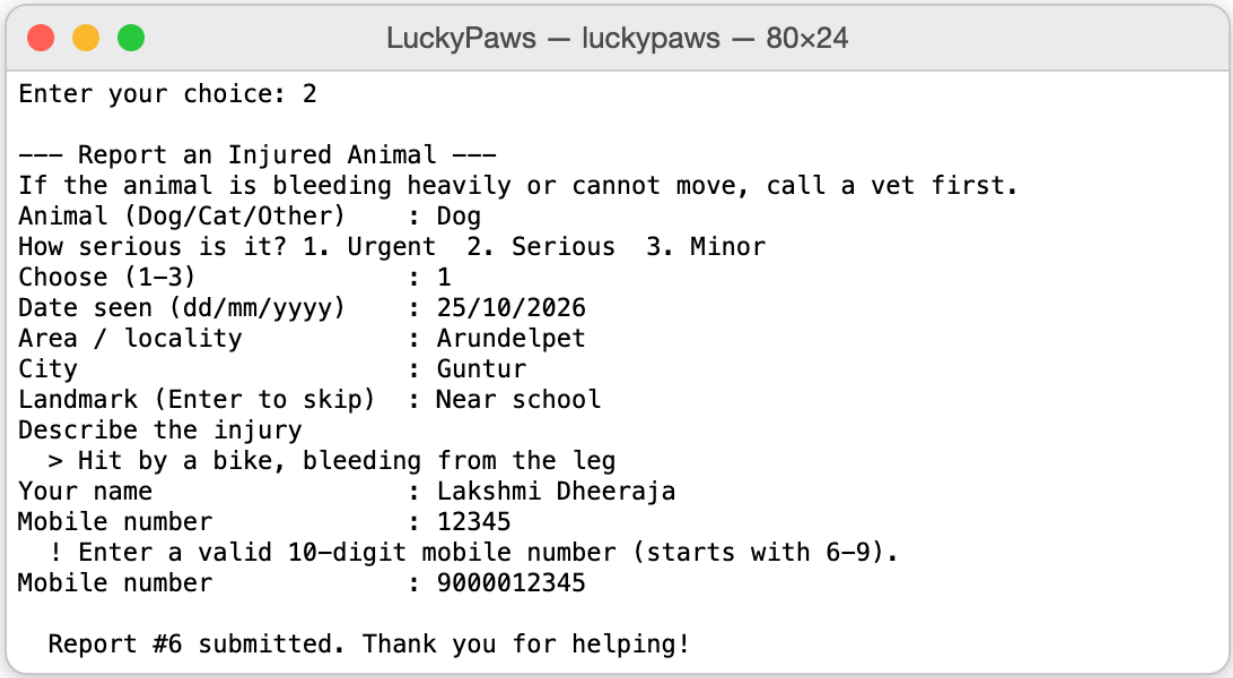
Output 1: Compiling and running the program



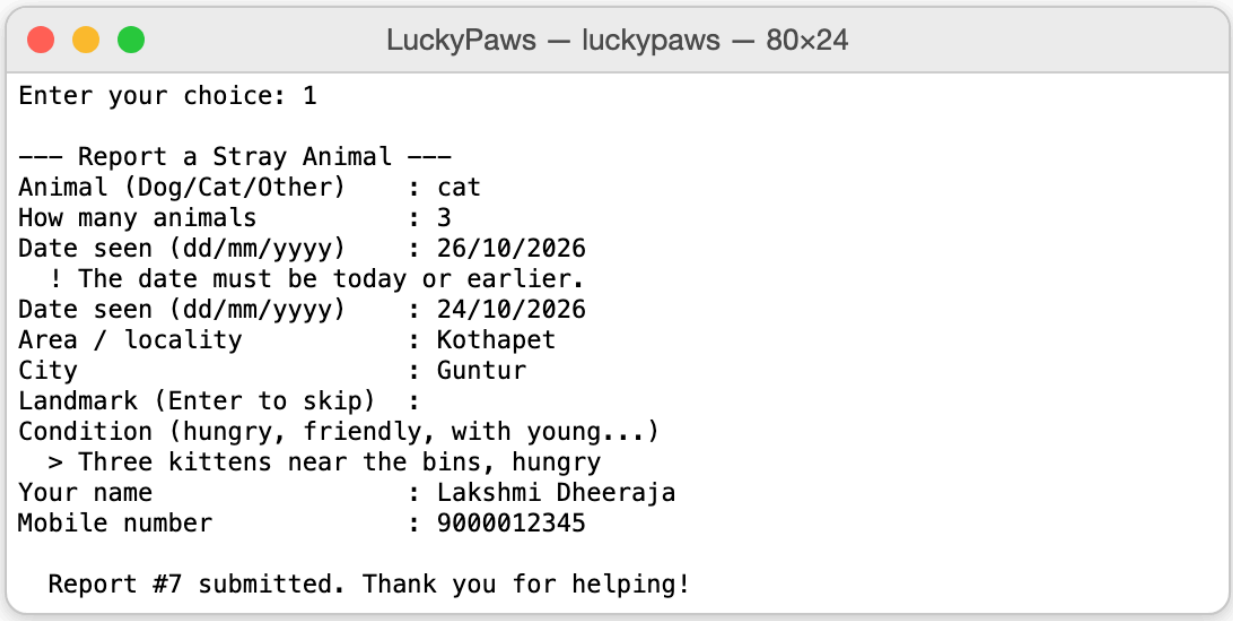
Output 2: Viewing all rescue and stray reports



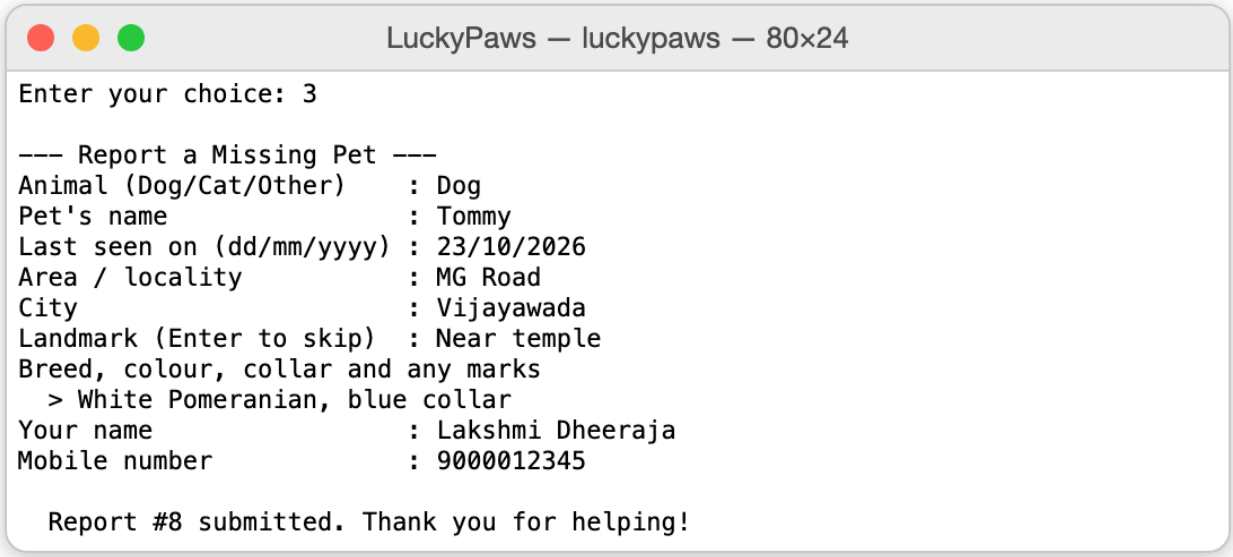
Output 3: Reporting an injured animal



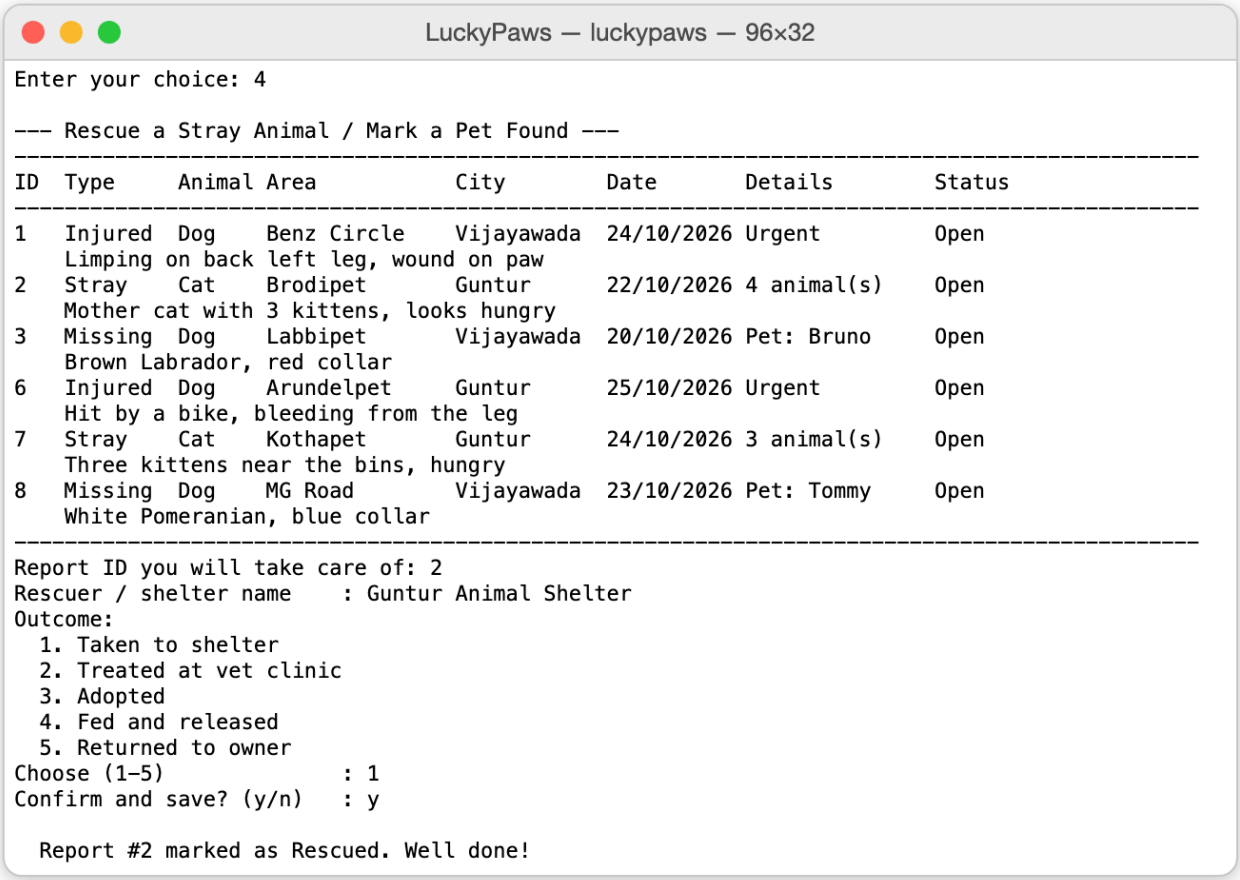
Output 4: Reporting a stray animal



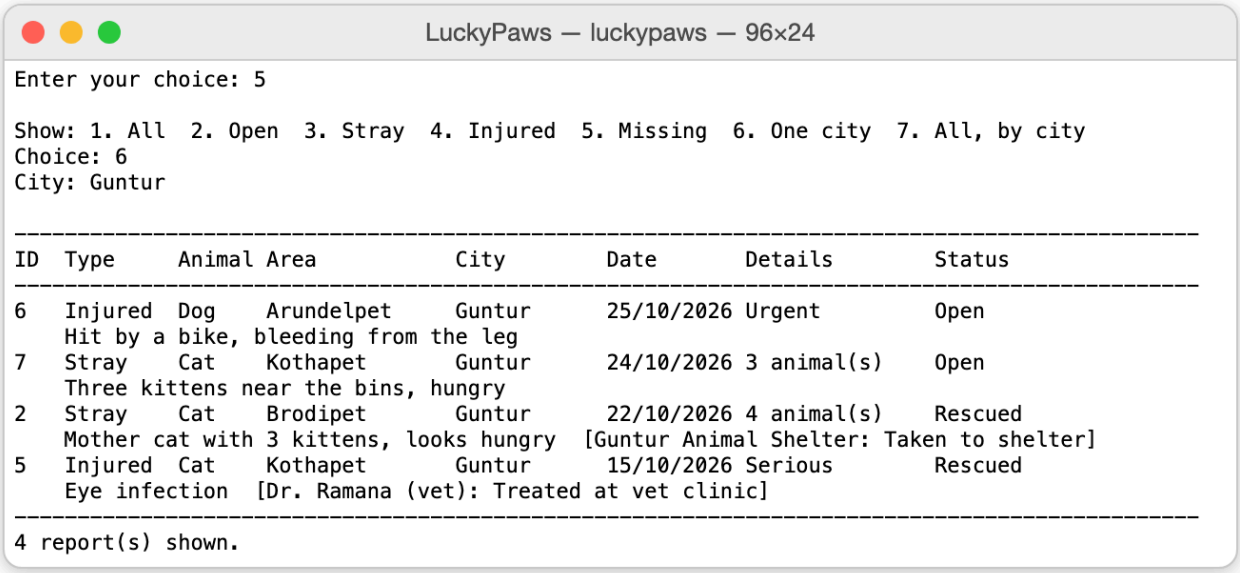
Output 5: Reporting a missing pet



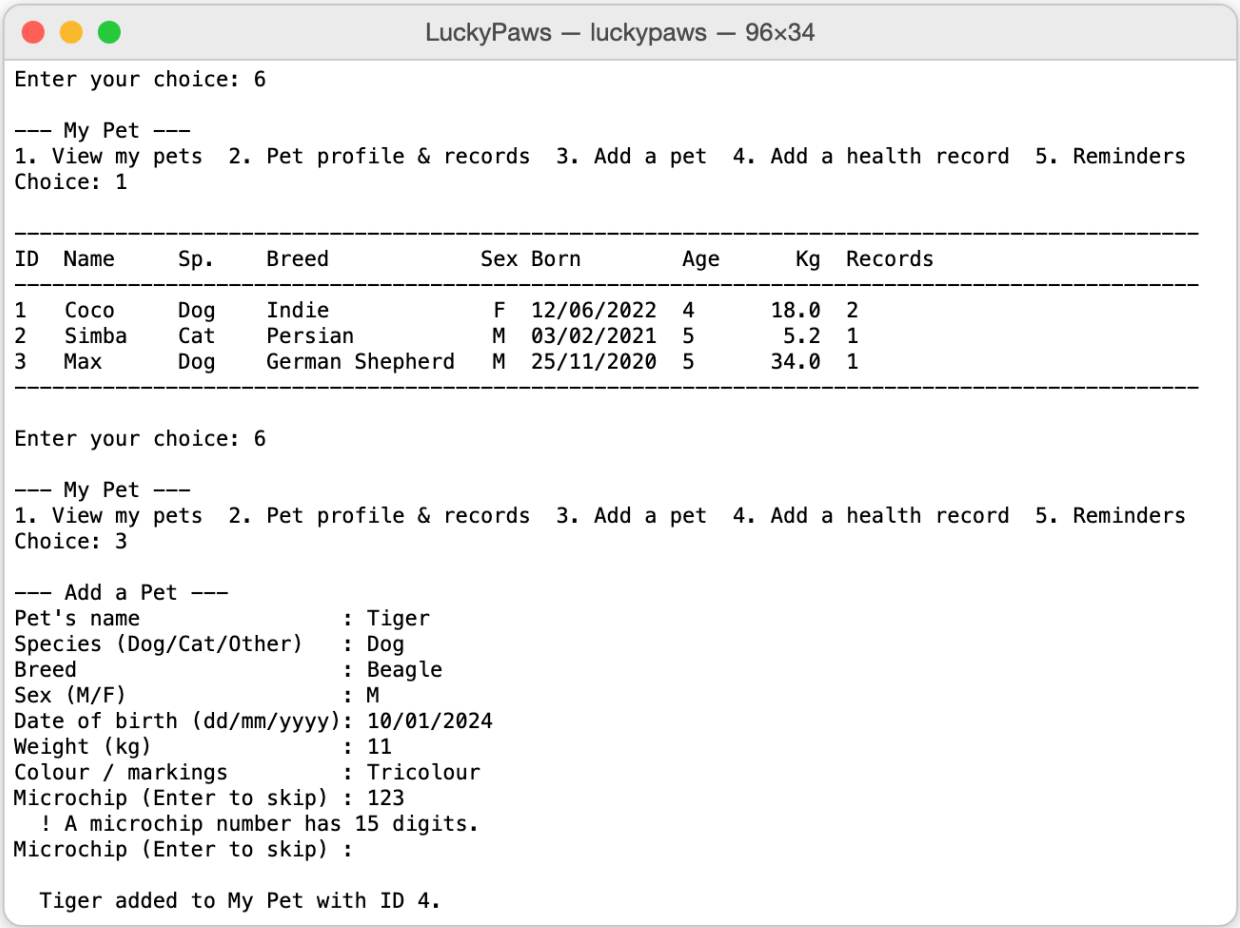
Output 6: Marking a stray animal as rescued



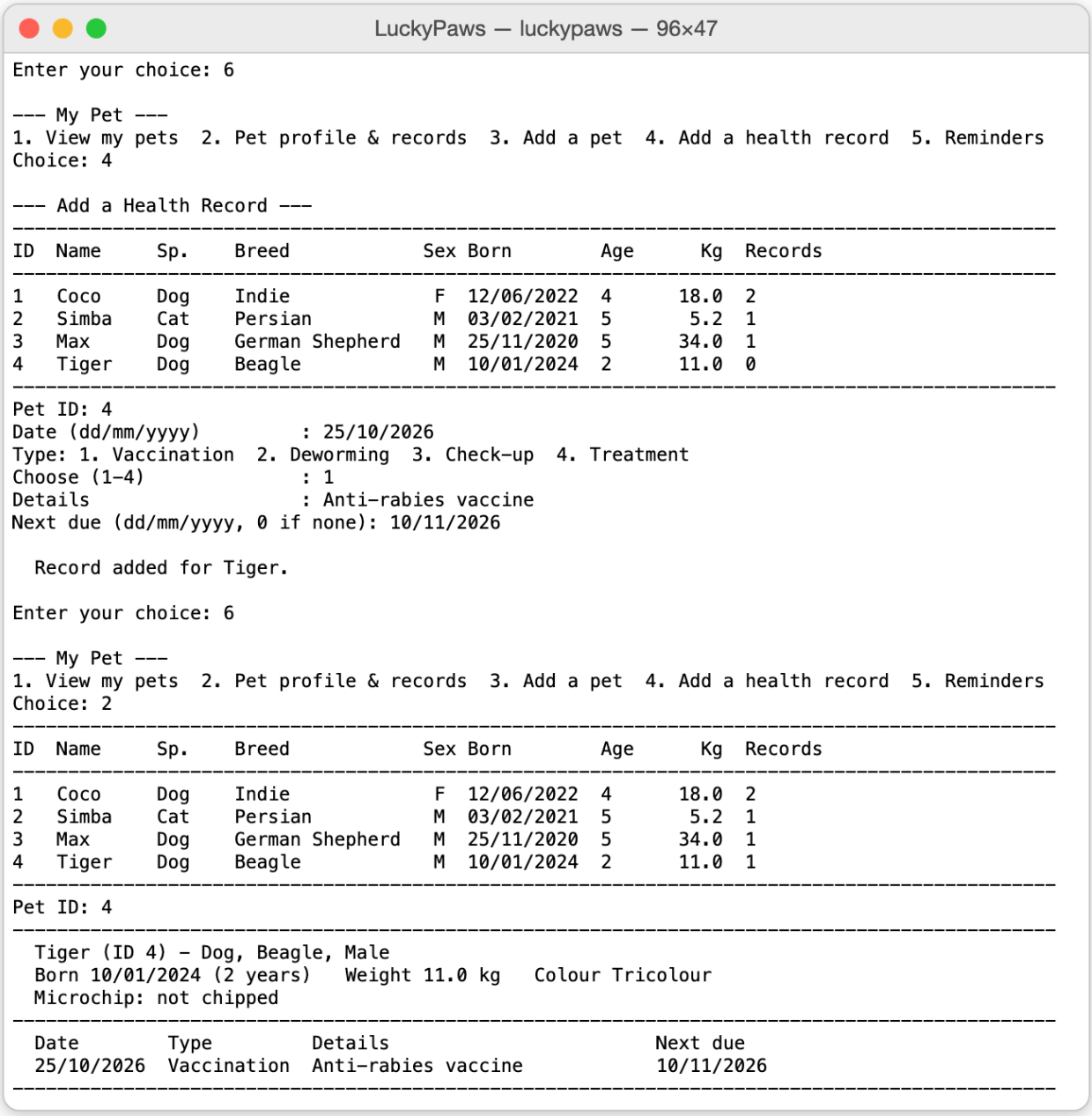
Output 7: Viewing reports of one city



Output 8: My Pet: list of pets and adding a new pet



Output 9: Adding a health record and viewing the pet profile



```
LuckyPaws — luckypaws — 96×47

Enter your choice: 6

--- My Pet ---
1. View my pets  2. Pet profile & records  3. Add a pet  4. Add a health record  5. Reminders
Choice: 4

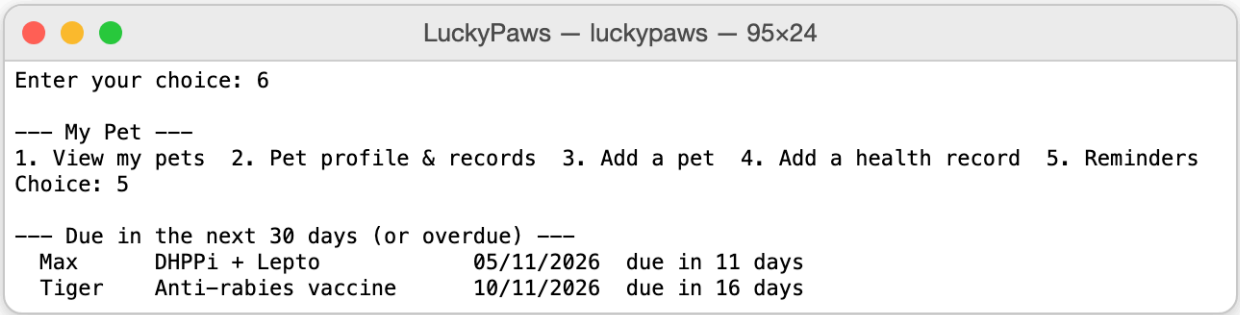
--- Add a Health Record ---
------------------------------------------------------------------------------------------------
ID  Name     Sp.    Breed            Sex Born        Age      Kg  Records
------------------------------------------------------------------------------------------------
1   Coco     Dog    Indie             F  12/06/2022  4      18.0  2
2   Simba    Cat    Persian           M  03/02/2021  5       5.2  1
3   Max      Dog    German Shepherd   M  25/11/2020  5      34.0  1
4   Tiger    Dog    Beagle            M  10/01/2024  2      11.0  0
------------------------------------------------------------------------------------------------
Pet ID: 4
Date (dd/mm/yyyy)         : 25/10/2026
Type: 1. Vaccination  2. Deworming  3. Check-up  4. Treatment
Choose (1-4)              : 1
Details                   : Anti-rabies vaccine
Next due (dd/mm/yyyy, 0 if none): 10/11/2026

  Record added for Tiger.

Enter your choice: 6

--- My Pet ---
1. View my pets  2. Pet profile & records  3. Add a pet  4. Add a health record  5. Reminders
Choice: 2
------------------------------------------------------------------------------------------------
ID  Name     Sp.    Breed            Sex Born        Age      Kg  Records
------------------------------------------------------------------------------------------------
1   Coco     Dog    Indie             F  12/06/2022  4      18.0  2
2   Simba    Cat    Persian           M  03/02/2021  5       5.2  1
3   Max      Dog    German Shepherd   M  25/11/2020  5      34.0  1
4   Tiger    Dog    Beagle            M  10/01/2024  2      11.0  1
------------------------------------------------------------------------------------------------
Pet ID: 4
------------------------------------------------------------------------------------------------
  Tiger (ID 4) - Dog, Beagle, Male
  Born 10/01/2024 (2 years)   Weight 11.0 kg   Colour Tricolour
  Microchip: not chipped
------------------------------------------------------------------------------------------------
  Date        Type         Details                        Next due
  25/10/2026  Vaccination  Anti-rabies vaccine            10/11/2026
------------------------------------------------------------------------------------------------
```

Output 10: Reminders for the next 30 days



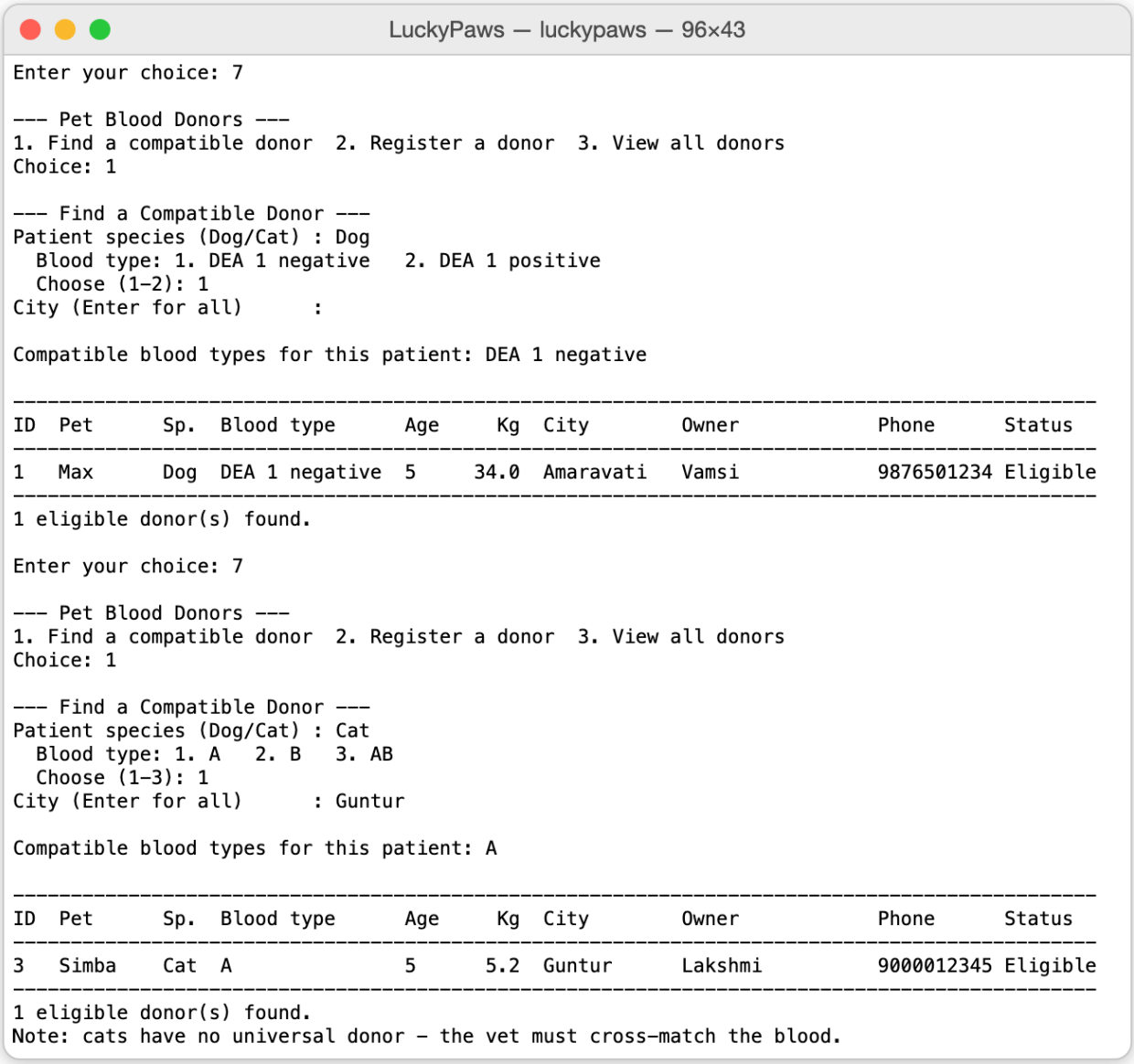
```
LuckyPaws — luckypaws — 95×24

Enter your choice: 6

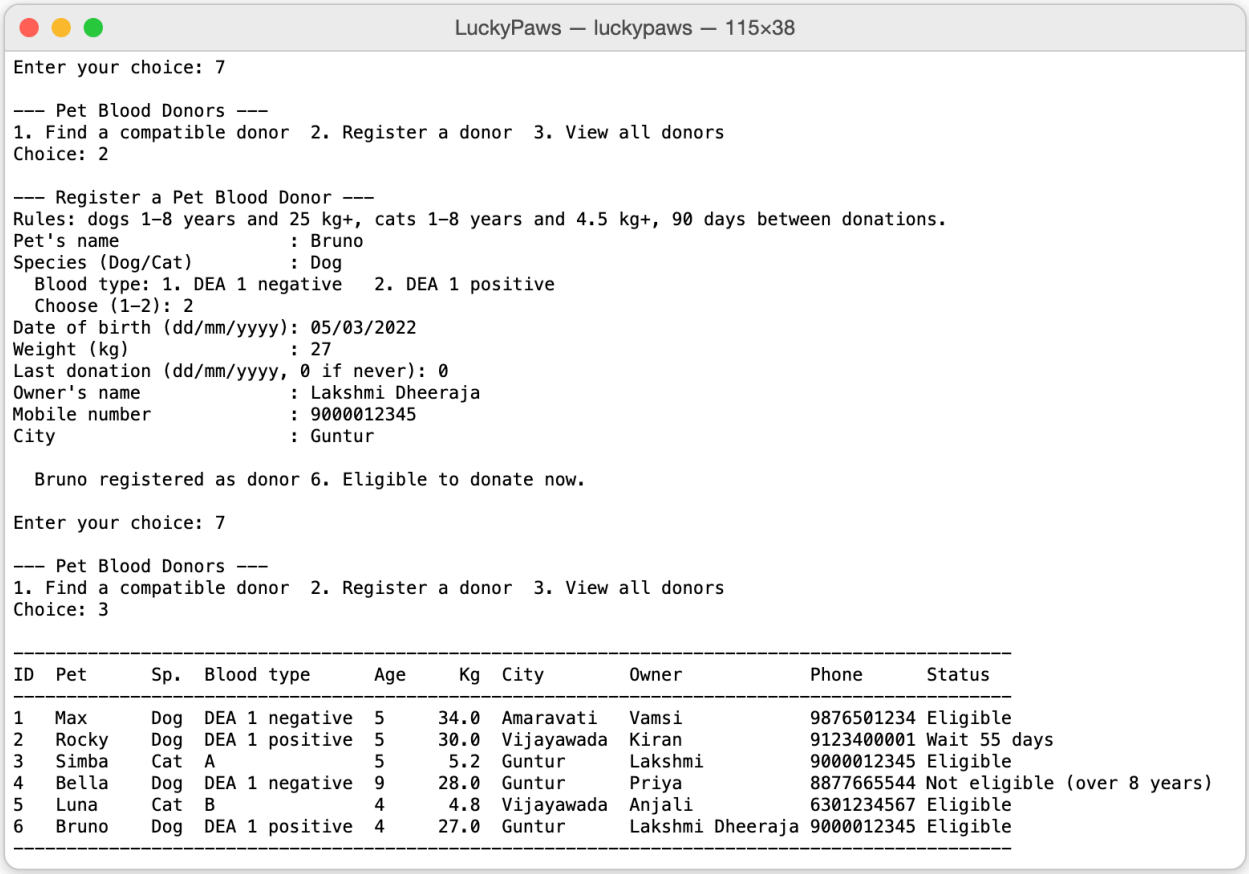
--- My Pet ---
1. View my pets  2. Pet profile & records  3. Add a pet  4. Add a health record  5. Reminders
Choice: 5

--- Due in the next 30 days (or overdue) ---
  Max      DHPPi + Lepto            05/11/2026  due in 11 days
  Tiger    Anti-rabies vaccine      10/11/2026  due in 16 days
```

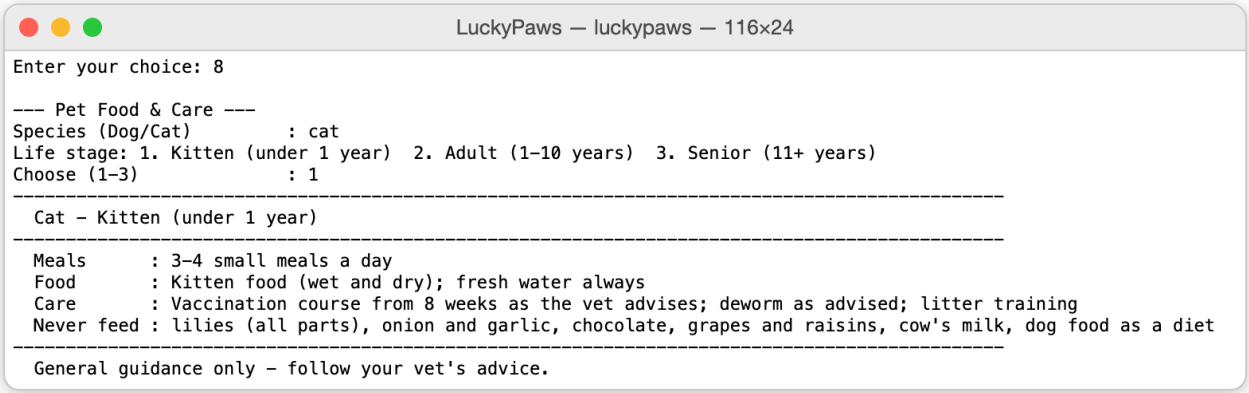
Output 11: Finding blood donors for a dog and a cat



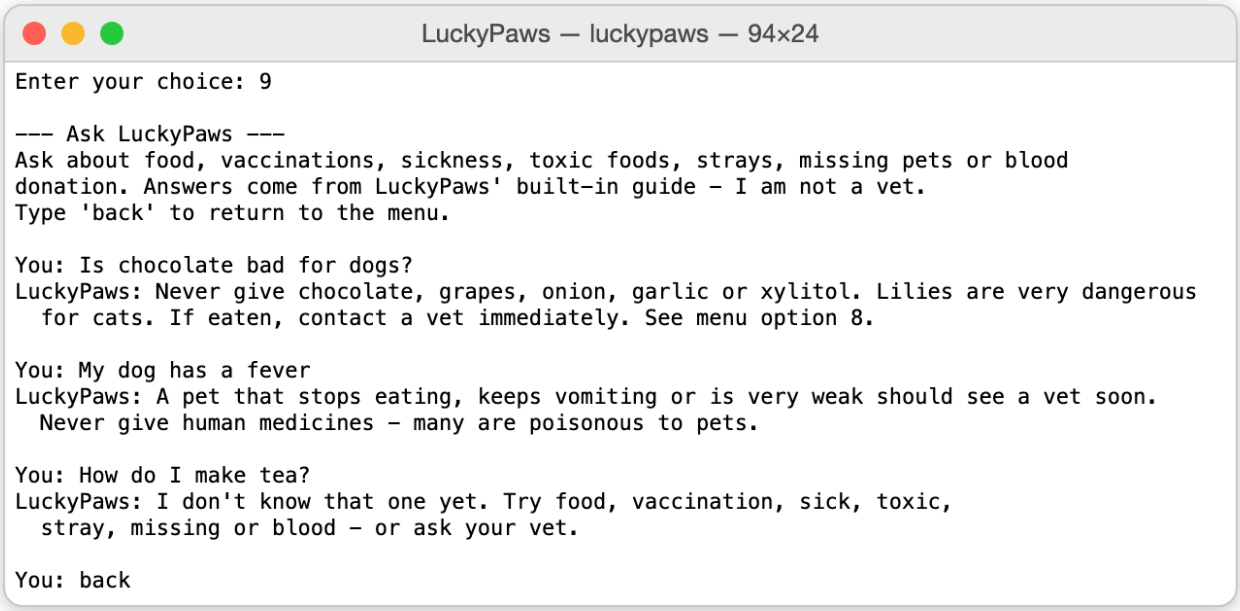
Output 12: Registering a blood donor and listing all donors



Output 13: Food and care guide



Output 14: Ask LuckyPaws



```
LuckyPaws — luckypaws — 94×24

Enter your choice: 9

--- Ask LuckyPaws ---
Ask about food, vaccinations, sickness, toxic foods, strays, missing pets or blood
donation. Answers come from LuckyPaws' built-in guide - I am not a vet.
Type 'back' to return to the menu.

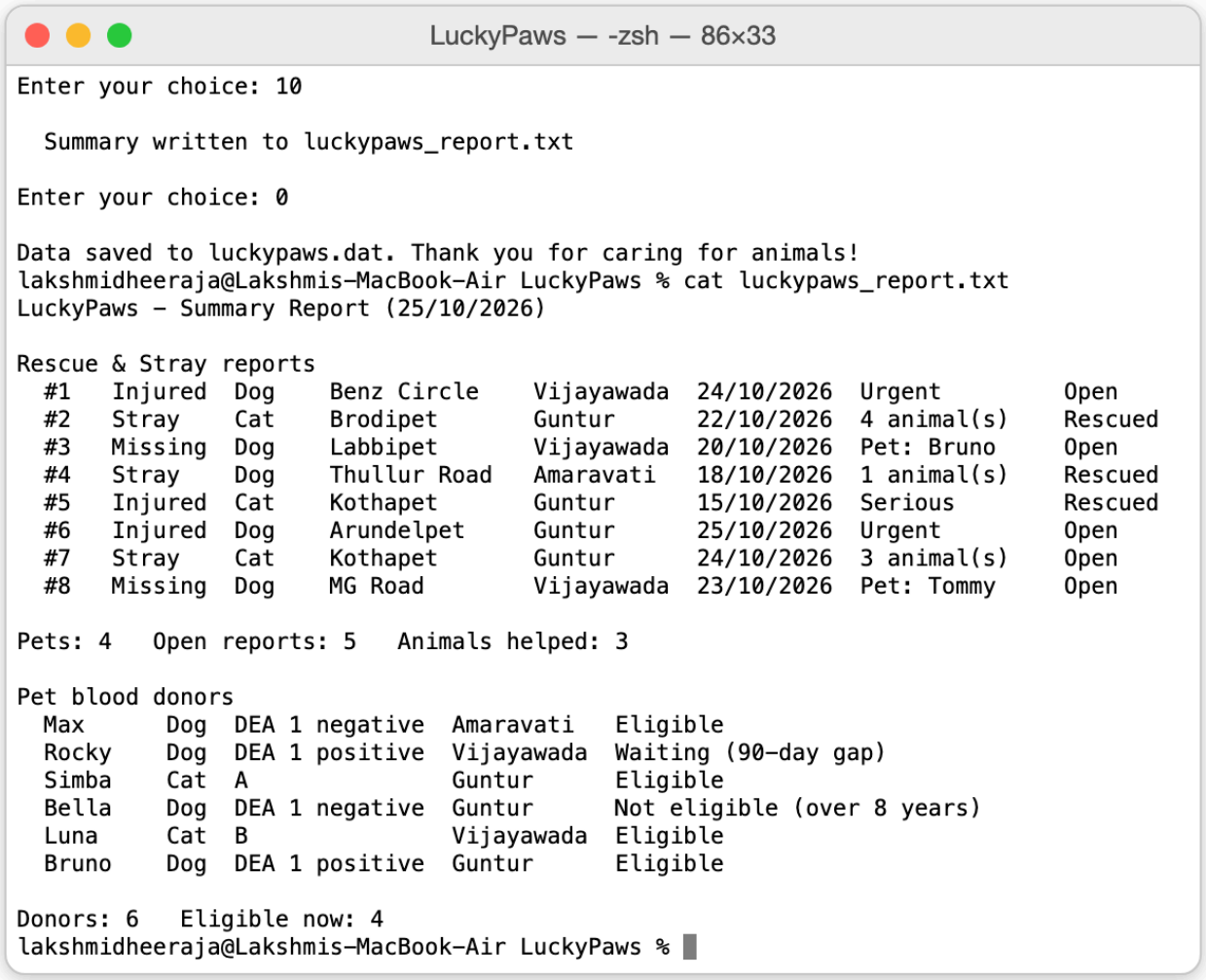
You: Is chocolate bad for dogs?
LuckyPaws: Never give chocolate, grapes, onion, garlic or xylitol. Lilies are very dangerous
  for cats. If eaten, contact a vet immediately. See menu option 8.

You: My dog has a fever
LuckyPaws: A pet that stops eating, keeps vomiting or is very weak should see a vet soon.
  Never give human medicines - many are poisonous to pets.

You: How do I make tea?
LuckyPaws: I don't know that one yet. Try food, vaccination, sick, toxic,
  stray, missing or blood - or ask your vet.

You: back
```

Output 15: Exporting the summary report to a text file



```
LuckyPaws — -zsh — 86×33

Enter your choice: 10

  Summary written to luckypaws_report.txt

Enter your choice: 0

Data saved to luckypaws.dat. Thank you for caring for animals!
lakshmidheeraja@Lakshmis-MacBook-Air LuckyPaws % cat luckypaws_report.txt
LuckyPaws - Summary Report (25/10/2026)

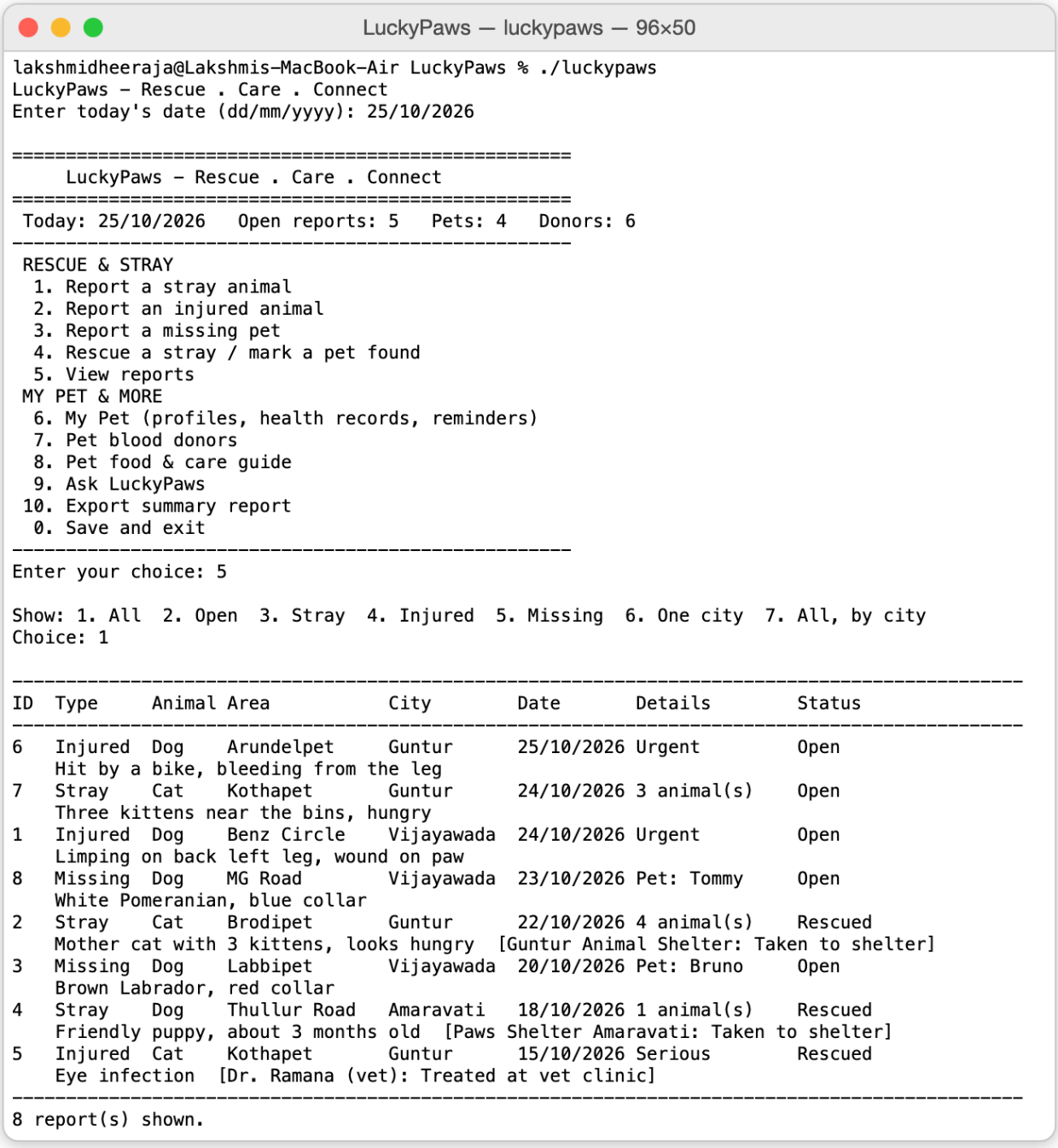
Rescue & Stray reports
  #1   Injured  Dog    Benz Circle    Vijayawada  24/10/2026  Urgent         Open
  #2   Stray    Cat    Brodipet       Guntur      22/10/2026  4 animal(s)    Rescued
  #3   Missing  Dog    Labbipet       Vijayawada  20/10/2026  Pet: Bruno     Open
  #4   Stray    Dog    Thullur Road   Amaravati   18/10/2026  1 animal(s)    Rescued
  #5   Injured  Cat    Kothapet       Guntur      15/10/2026  Serious        Rescued
  #6   Injured  Dog    Arundelpet     Guntur      25/10/2026  Urgent         Open
  #7   Stray    Cat    Kothapet       Guntur      24/10/2026  3 animal(s)    Open
  #8   Missing  Dog    MG Road        Vijayawada  23/10/2026  Pet: Tommy     Open

Pets: 4   Open reports: 5   Animals helped: 3

Pet blood donors
  Max      Dog  DEA 1 negative  Amaravati   Eligible
  Rocky    Dog  DEA 1 positive  Vijayawada  Waiting (90-day gap)
  Simba    Cat  A               Guntur      Eligible
  Bella    Dog  DEA 1 negative  Guntur      Not eligible (over 8 years)
  Luna     Cat  B               Vijayawada  Eligible
  Bruno    Dog  DEA 1 positive  Guntur      Eligible

Donors: 6   Eligible now: 4
lakshmidheeraja@Lakshmis-MacBook-Air LuckyPaws %
```

Output 16: Running the program again (data is saved in the file)



```
LuckyPaws — luckypaws — 96×50

lakshmidheeraja@Lakshmis-MacBook-Air LuckyPaws % ./luckypaws
LuckyPaws - Rescue . Care . Connect
Enter today's date (dd/mm/yyyy): 25/10/2026

==================================================
     LuckyPaws - Rescue . Care . Connect
==================================================
 Today: 25/10/2026   Open reports: 5   Pets: 4   Donors: 6
--------------------------------------------------
 RESCUE & STRAY
  1. Report a stray animal
  2. Report an injured animal
  3. Report a missing pet
  4. Rescue a stray / mark a pet found
  5. View reports
 MY PET & MORE
  6. My Pet (profiles, health records, reminders)
  7. Pet blood donors
  8. Pet food & care guide
  9. Ask LuckyPaws
 10. Export summary report
  0. Save and exit
--------------------------------------------------
Enter your choice: 5

Show: 1. All  2. Open  3. Stray  4. Injured  5. Missing  6. One city  7. All, by city
Choice: 1

------------------------------------------------------------------------------------------------------
ID  Type     Animal Area           City        Date       Details        Status
------------------------------------------------------------------------------------------------------
6   Injured  Dog    Arundelpet     Guntur      25/10/2026 Urgent         Open
    Hit by a bike, bleeding from the leg
7   Stray    Cat    Kothapet       Guntur      24/10/2026 3 animal(s)    Open
    Three kittens near the bins, hungry
1   Injured  Dog    Benz Circle    Vijayawada  24/10/2026 Urgent         Open
    Limping on back left leg, wound on paw
8   Missing  Dog    MG Road        Vijayawada  23/10/2026 Pet: Tommy     Open
    White Pomeranian, blue collar
2   Stray    Cat    Brodipet       Guntur      22/10/2026 4 animal(s)    Rescued
    Mother cat with 3 kittens, looks hungry  [Guntur Animal Shelter: Taken to shelter]
3   Missing  Dog    Labbipet       Vijayawada  20/10/2026 Pet: Bruno     Open
    Brown Labrador, red collar
4   Stray    Dog    Thullur Road   Amaravati   18/10/2026 1 animal(s)    Rescued
    Friendly puppy, about 3 months old  [Paws Shelter Amaravati: Taken to shelter]
5   Injured  Cat    Kothapet       Guntur      15/10/2026 Serious        Rescued
    Eye infection  [Dr. Ramana (vet): Treated at vet clinic]
------------------------------------------------------------------------------------------------------
8 report(s) shown.
```